

高性能芯片热管理系统优化

摘要

针对高性能芯片歧管式微通道液冷结构中散热能力、泵功消耗与温度均匀性相互制约的问题,本文以针肋宽度比、歧管深高比和针肋排数为设计变量,以热阻、压降和温度非均匀性为评价指标,构建了**机理建模、代理预测、多目标寻优、偏好鲁棒设计与扰动评估**相衔接的求解框架。

对**问题一**,依据质量守恒、动量守恒、能量守恒及固体导热方程建立流固耦合控制模型,并给出三项性能指标的统一定义。通过无量纲化提取雷诺数、普朗特数和结构参数比等主导因素,再依据达西摩擦、局部阻力叠加及热阻串联机理构造**压降幂律关联式与热阻半经验关联式**,结合控制变量分析揭示各结构参数对流动阻力、换热能力和温度均匀性的作用方向及其耦合关系。

对**问题二**,对结构参数进行标准化处理,分别以三项性能指标为输出建立 **Constant+RBF+White 复合核高斯过程回归模型**,形成多输入-单输出代理映射。采用留出法划分训练集与测试集,并以决定系数、均方根误差和平均绝对百分比误差检验拟合与泛化能力,从而获得可用于后续寻优和不确定性传播的快速性能评价函数。

对**问题三**,将高斯过程代理模型作为目标函数,在物理加工约束和样本支撑域内建立热阻、压降与温度非均匀性同时最小化的多目标优化模型。采用 **NSGA-II 全局搜索、L-BFGS-B 局部精修与整数离散修正**相结合的混合策略求取 Pareto 前沿,再经 Min-Max 无量纲化和均等权重理想点法完成综合决策,得到兼顾三项性能的设计方案,并通过代理模型回代检验其可行性与自洽性。

对**问题四**,在指标权重单纯形上设置代表性偏好场景,建立由偏好权重到最优结构参数的敏感性映射。以各场景定制最优方案为基准定义后悔值,进而采用**最小最大后悔值准则**构建鲁棒优化模型,使候选方案在最不利偏好下的综合性能损失尽可能小;同时结合最大后悔值与综合性能变异系数评价方案对权重变化的稳定性。

对**问题五**,将加工误差与运行工况波动分层处理。结构参数层由代理模型计算局部归一化敏感性系数和 Sobol 全局敏感性指数,工况参数层依据无量纲标度关系进行小扰动一阶估计;随后采用拉丁超立方采样与蒙特卡洛传播获得性能响应分布,以变异系数、置信上限和越界失效概率判定稳定性。该方法识别了需要重点控制的加工参数,并表明所选方案在设定的小扰动范围内具有较好的工程稳定性。

关键字: 歧管式微通道; 高斯过程回归; 多目标优化; 最小最大后悔值; Sobol 敏感性分析

一、问题重述

1.1 问题背景

随着高性能芯片的集成度和功率密度不断提高, 散热已成为影响芯片性能、寿命及封装可靠性的关键因素。散热不及时会形成局部热点, 引起结温升高、性能下降甚至器件失效, 因此需要在有限空间和能耗下实现高效、均匀的散热。

本文研究歧管式微通道液冷热管理系统 [1]。系统采用去离子水冷却、氮化铝衬底, 通过歧管分配层和针肋强化换热, 改善传统微通道压降较大、温度分布不均等问题。芯片散热区域为 $10 \times 10 \text{ mm}^2$ 的方形区域, 内部布置多条平行微通道, 并在各歧管单元对应区域内设置针肋强化结构; 冷却液由顶部入口经歧管层分配后流入微通道换热层, 最终从侧部出口流出。主要结构参数为针肋宽度比、歧管深高比和针肋排数, 性能指标为热阻、压降及温度非均匀性。

1.2 问题提出

1.2.1 问题一

首先, 结合传热学和流体力学的基本原理, 分析冷却液在歧管分配层和微通道换热层中的流动、换热及芯片导热过程。以针肋宽度比、歧管深高比和针肋排数为输入参数, 以热阻、压降和温度非均匀性为输出指标, 建立结构参数与系统性能之间的机理模型。在此基础上, 分别讨论各结构参数对三项指标的影响方向和变化规律, 分析散热能力、流动阻力与温度均匀性之间可能存在的制约关系, 并说明选取三项指标进行综合评价的合理性。

1.2.2 问题二

附件 2 给出了不同结构参数组合下系统性能指标的样本数据。利用这些数据, 以三项结构参数作为模型输入, 以热阻、压降和温度非均匀性作为模型输出, 建立能够近似替代复杂数值计算或实验过程的代理模型, 刻画结构参数与性能指标之间的非线性映射关系。同时, 通过训练集与测试集划分、交叉验证或误差指标等方法, 检验模型的拟合精度、泛化能力和预测稳定性, 为后续优化计算提供可靠的性能评价工具。

1.2.3 问题三

在问题二代理模型的基础上, 将针肋宽度比、歧管深高比和针肋排数作为待确定的设计变量, 以热阻、压降和温度非均匀性同时最小化作为优化目标, 并将附件给出的参数

范围作为可行域约束,建立多目标优化模型。由于降低热阻、减小压降和改善温度均匀性之间可能存在冲突,需要结合指标无量纲化、权重设置或综合评价准则,从候选解中筛选出兼顾各项性能的综合最优方案,并给出相应的结构参数取值。

1.2.4 问题四

不同应用场景对散热能力、泵功消耗和温度均匀性的重视程度不同,因此需要研究三项指标权重变化对最优设计方案的影响。通过设置具有代表性的权重组合,比较不同偏好下最优结构参数、目标函数值及方案排序的变化,判断优化结果对权重扰动的敏感程度。进一步从候选方案中选取在较大权重范围内性能波动较小、排名稳定或综合损失较低的鲁棒设计方案,并说明其判定标准以及优势。

1.2.5 问题五

实际制造过程中不可避免会产生尺寸加工误差,系统运行时也可能受到流量、温度等工况波动的影响,使实际结构参数和工作状态偏离设计值。针对这些因素,在最优方案附近对关键参数进行小范围单因素或多因素扰动,观察热阻、压降和温度非均匀性的变化幅度,计算相应的敏感性指标,识别对系统性能影响较大的关键因素。最后,通过比较扰动前后的性能变化和方案稳定程度,评估设计方案的敏感性与稳定性。

二、问题分析

2.1 问题一的分析

本文基于传热学与流体力学理论 [2, 3] 建立流固耦合控制方程组,定义热阻、压降与温度非均匀性三项性能指标;进而提取雷诺数、普朗特数与结构参数比等主导无量纲量,确认附件数据处于层流工况,为后续代理模型输入选取提供机理依据;在此基础上,依据达西摩擦、局部阻力叠加与热阻串联机理构造压降幂律关联式与热阻二次结构关联式,由附件数据最小二乘标定;最后固定其余参数,定量统计各结构参数对三项指标的影响方向与幅度,验证机理规律并论证三项指标综合评价的合理性。

2.2 问题二的分析

对 84 组样本按 8 : 2 留出法划分训练/测试集,输入特征做 Z-score 标准化;考虑到样本量小且映射高度非线性,选用高斯过程回归 [4],以 $\text{Constant} \times \text{RBF} + \text{White}$ 复合核建立多输入-单输出 (MISO) 映射,用极大似然估计优化超参数;最后以决定系数、均方根误差与平均绝对百分比误差评估拟合与泛化能力,确认代理模型可替代 CFD 求解器。

2.3 问题三的分析

以 GPR 代理模型为目标函数, 在样本支撑域内建立三指标同时最小化的多目标优化模型 (热阻-压降负相关 $r = -0.684$ 表明解集为 Pareto 前沿); 采用 NSGA-II[6] 全局搜索 Pareto 前沿, L-BFGS-B[7, 8] 局部精修, 并对排数整数约束离散修正; 最终以 Min-Max 无量纲化消除量纲差异, 用均等权重理想点法 [5] 将多目标转化为贴近理想解的单目标决策, 唯一确定综合最优设计方案并回代验证。

2.4 问题四的分析

在权重单纯形上采样全场景偏好, 建立由偏好权重到最优结构参数的敏感性映射, 量化权重波动对方案的驱动作用; 以各场景定制最优为基准定义后悔值, 采用最小最大后悔值准则 [9] 求取最坏偏好下综合性能损失最小的鲁棒方案; 最后以权重空间变异系数与最大后悔值对比鲁棒方案与等权综合方案, 论证鲁棒设计的工程价值。

2.5 问题五的分析

对加工误差 (结构参数) 与工况波动 (流量、入口温度、发热功率) 两类扰动进行概率分布表征, 建立 $\pm 5\%$ 综合摄动模型; 结构参数层由代理模型计算局部归一化敏感性系数与 Sobol 全局敏感性指数 [10], 工况参数层基于无量纲标度关系一阶近似; 采用拉丁超立方采样与蒙特卡洛传播 [11] 生成性能响应分布, 以变异系数、95% 置信上限与越界失效概率判定稳定性; 最后将综合最优方案与鲁棒方案一并代入扰动传播, 对比评估两类方案的工程稳定性。

三、模型假设

1. 冷却介质 (去离子水) 为不可压缩、稳态流动, 物性参数按平均温度取值;
2. 忽略黏性耗散与压缩功, 流体能量方程中不计黏性产热项 (式 3 不含耗散项);
3. 流动状态按附件数据中的雷诺数判定为层流或湍流 (本文问题一按层流建模);
4. 忽略辐射换热, 换热仅由强制对流主导;
5. 芯片热源视为均匀体热源, 强度为 $5 \times 10^9 \text{ W/m}^3$, 忽略边缘效应, 体热源仅作用于芯片热源区域 (见式 4);
6. 忽略芯片、衬底与冷却液之间的界面接触热阻, 认为各界面传热连续;
7. 问题一机理分析中, 同一歧管单元内各通道间的微观流量分配差异忽略不计, 流量分配均匀性的宏观效应仅通过歧管深高比 β 等结构参数刻画;
8. 结构参数与工况仅在附件给定的设计空间 (样本支撑范围) 内取值, 代理模型在该范围内视为具有足够精度、可用于替代仿真评估, 超出该范围的外推预测不可靠。

四、符号说明

表 1 主要符号说明 (一): 传热与流动物理量

符号	含义	单位
α	针肋宽度比 (针肋直径/通道宽度)	—
β	歧管深高比 (歧管层/微通道层深高比)	—
n	单个歧管单元内沿流向的针肋排数	—
R_{th}	总热阻	K/W
ΔP	压降	Pa
ΔT	温度非均匀性 (最高-最低温度差)	K
σ_T	温度非均匀性 (底面温度标准差, 备选)	K
T_{max}	芯片表面最高温度	K
T_{min}	芯片表面最低温度	K
T_{in}	冷却液入口温度 (293 K)	K
T_f	流体温度	K
T_s	固体温度	K
P_{in}	入口压力	Pa
P_{out}	出口压力	Pa
Q	芯片总发热功率	W
$\vec{v}$	速度矢量	m/s
h	对流换热系数	W/(m ² ·K)
A	换热面积	m ²
D_h	通道水力直径	m
u	流速	m/s
ρ	冷却液密度	kg/m ³
C_p	冷却液比热容	J/(kg·K)
μ	冷却液动力黏度	Pa·s
k_f	流体导热系数	W/(m·K)
k_s	固体 (衬底) 导热系数	W/(m·K)
f	沿程阻力系数	—
Re	雷诺数	—
Pr	普朗特数	—

表 2 主要符号说明 (二): 代理模型量

符号	含义	单位
$\mathbf{X}$	结构参数输入向量 $[x_1, x_2, x_3]^T$	—
$\mathbf{Y}$	性能指标输出向量 $[y_1, y_2, y_3]^T$	—
$\tilde{x}_i$	标准化后的第 i 个输入特征	—
μ_i	第 i 个特征在训练集上的均值	—
σ_i	第 i 个特征在训练集上的标准差	—
$f_j(\cdot)$	第 j 项性能指标的潜在函数	—
$\mathcal{GP}$	高斯过程	—
$k(\cdot, \cdot)$	核函数	—
σ_f^2	信号方差 (核函数幅值)	—
l_i	第 i 个结构参数的各向异性长度尺度	—
σ_n^2	观测噪声方差	—
$\delta(\cdot, \cdot)$	Kronecker delta 函数	—
$\boldsymbol{\theta}$	超参数集合 $[\sigma_f, l_1, l_2, l_3, \sigma_n]$	—
$\mathbf{K}$	训练集核矩阵	—
$\mathbf{k}$	训练集与新样本间的协方差向量	—
$\mathbf{I}$	单位矩阵	—
$\mathbf{y}_{train}$	训练集输出向量	—
R^2	决定系数	—
$RMSE$	均方根误差	—
$MAPE$	平均绝对百分比误差	—
$\hat{y}_k$	第 k 个样本的模型预测值	—
$\bar{y}$	观测均值	—
N	样本数	—

表 3 主要符号说明 (三): 优化模型量

符号	含义	单位
$f_{GPR,j}(\mathbf{X})$	第 j 项指标的 GPR 代理模型预测函数	—
$\mathbf{F}(\mathbf{X})$	多目标向量最小化函数 $[f_{Rth}, f_{\Delta P}, f_{\Delta T}]^T$	—
$\tilde{f}_j(\mathbf{X})$	归一化后的第 j 项指标	—
$f_{j,\min}/f_{j,\max}$	第 j 项指标的极值边界 (取自附件 2)	—
$\mathbf{O}^*$	理想最佳状态点 $(0, 0, 0)^T$	—
w_j	第 j 项指标的权重	—
$D(\mathbf{X})$	综合优化目标函数 (加权欧氏距离)	—
$\mathbb{Z}^+$	正整数集	—
$\mathbf{F}(\mathbf{X})$	无量纲性能指标矢量 $[\tilde{f}_1, \tilde{f}_2, \tilde{f}_3]^T$	—
$\mathbf{W}$	场景偏好权重矢量 $[w_1, w_2, w_3]^T$	—
Δ^2	二维标准单纯形 (全场景偏好空间)	—
$D_{\mathbf{W}}(\mathbf{X})$	加权理想点目标函数	—
$\Omega_{\mathbf{X}}$	决策变量可行域	—
$\mathbf{X}^*(\mathbf{W})$	偏好 $\mathbf{W}$ 下的最优设计	—
$\mathbf{S}_{\mathbf{X},\mathbf{W}}$	决策变量关于权重的梯度敏感性矩阵	—
α^*	最优针肋宽度比 (热阻极小值对应点)	—
$\mathcal{P}$	工况参数集合 $\{\dot{m}, T_{in}, q_v\}$	—
$R(\mathbf{X}, \mathbf{W})$	方案 $\mathbf{X}$ 在场景 $\mathbf{W}$ 下的后悔值	—
$CV(\mathbf{X})$	综合性能变异系数 (问题四, 权重空间)	—
$\mu_{\mathbf{W}}, \sigma_{\mathbf{W}}$	目标值在全偏好空间的期望与标准差	—
R_{max}	最大后悔值	—
ϵ	后悔值容许上限	—
$\nabla_{\mathbf{W}}$	关于权重矢量的梯度算子	—

表 4 主要符号说明 (四): 扰动敏感性与稳定性分析量

符号	含义	单位
$\mathbf{Z}$	综合摄动向量 $[\mathbf{X}, \mathbf{P}]^T \in \mathbb{R}^6$	—
$\mathbf{P}$	工况参数矢量 $[\dot{m}, T_{in}, q_v]^T$	—
$\dot{m}$	入口质量流量 (基准 1 g/s)	g/s
q_v	芯片发热功率 (基准 5×10^9 W/m ³)	W/m ³
$\delta_{\mathbf{Z}}$	摄动向量	—
$\Sigma_{\mathbf{Z}}$	摄动协方差矩阵 (对角阵)	—
$\sigma_{x_i}^2$	第 i 个结构参数的扰动方差	—
$\sigma_{\dot{m}}^2, \sigma_{T_{in}}^2, \sigma_{q_v}^2$	工况参数扰动方差	—
$\mathcal{N}(\mu, \sigma^2)$	正态分布	—
$\mathcal{U}[-\delta, +\delta]$	有界均匀分布	—
ε	相对扰动幅度 (如 1%~5%)	—
$G_j(\mathbf{Z})$	扩展后的代理模型响应映射函数	—
S_{ij}^{local}	局部归一化敏感性系数	—
$z_{i,0}$	第 i 个输入的标称值	—
$y_{j,0}$	第 j 项指标的标称值	—
$V(y_j)$	第 j 项指标的总方差	—
S_i^j	一阶主效应 (Sobol) 指数	—
S_{Ti}^j	总效应 (Sobol) 指数	—
$\mathbf{Z}_{-i}$	除 z_i 外的其余变量集合	—
V_{im}^j	两参数交互方差项	—
CV_j^{pert}	第 j 项指标的扰动变异系数 (问题五, 扰动空间)	—
μ_{y_j}, σ_{y_j}	指标均值与标准差	—
$y_{j,P95}$	95% 置信区间波动上限	—
$y_{j,max}^{limit}$	工程许可的最大性能恶化容限	—
$P_{fail,j}$	越界失效概率	—
$\mathbb{I}(\cdot)$	示性函数	—

五、模型的建立与求解

5.1 问题一: 传热与流动基本机理模型的建立与求解

5.1.1 控制方程

基于第 3 节的模型假设, 以计算流体力学与传热学理论 [2, 3] 为依据, 对歧管式微通道内冷却介质的流动与换热过程建立控制方程组。计算域包含去离子水冷却液 (按附件 1: $\rho = 998.2$ kg/m³、 $C_p = 4182$ J/(kg·K)、 $k_f = 0.6$ W/(m·K)、 $\mu = 1.003 \times 10^{-3}$ Pa·s) 与氮化铝衬底 ($k_s = 200$ W/(m·K)); 固体域外侧与环境 (293 K) 以自然对流 (换热系数 10 W/(m²·K)) 换热。控制方程组如下:

(1) 连续性方程 (质量守恒):

$$\nabla \cdot \vec{v} = 0, \quad (1)$$

(2) 动量守恒方程 (Navier–Stokes 方程):

$$\rho(\vec{v} \cdot \nabla)\vec{v} = -\nabla P + \mu \nabla^2 \vec{v}, \quad (2)$$

(3) 能量守恒方程 (流体对流换热):

$$\rho C_p(\vec{v} \cdot \nabla)T_f = k_f \nabla^2 T_f, \quad (3)$$

(4) 能量守恒方程 (固体导热):

$$k_s \nabla^2 T_s + q_v = 0, \quad (4)$$

其中 q_v 为体热源强度, 仅在芯片热源区域内非零 (见假设 5), 衬底与流体区域内取零。

其中式 (1)–(4) 分别刻画了冷却液的质量、动量与能量输运过程以及衬底内部的稳态导热过程, 构成问题一机理分析的基础 [12]。

5.1.2 三项性能指标的数学定义

在上述控制方程组所描述的流动换热过程基础上, 定义系统的三项关键性能指标:

(1) 热阻 R_{th} 反映系统将热量从芯片导出至冷却液的能力:

$$R_{th} = \frac{T_{max} - T_{in}}{Q}, \quad (5)$$

其中 T_{max} 为芯片表面最高温度, T_{in} 为入口温度 (293 K), Q 为芯片总发热功率。

(2) 压降 ΔP 反映冷却液流经系统全程的阻力特性:

$$\Delta P = P_{in} - P_{out}. \quad (6)$$

(3) 温度非均匀性 ΔT 反映芯片表面温度分布的均匀程度:

$$\Delta T = T_{max} - T_{min}, \quad (7)$$

其中 T_{min} 为芯片表面最低温度。亦可用底面温度分布的标准差 σ_T 表征非均匀性, 两种定义在本文后续分析中视情况选用。

5.1.3 关键结构参数对性能指标的影响规律

针肋宽度比 α 增大时, 针肋换热面积增加、绕流扰动增强, 对流换热系数提高, 热阻 R_{th} 下降; 但针肋同时挤占流通截面, 超过临界值后肋间流道过窄、流动阻滞加剧, 热阻

转为回升,呈”先降后升”的非单调变化,最优宽度比在 $\alpha \approx 0.2$ 附近 (定量结果见 5.1.5 节); 压降 ΔP 则因摩擦与形体阻力增大而总体升高,排数较多时增幅更显著。

歧管深高比 β 增大时,分配通道流通截面增大、沿程阻力降低,压降总体减小,流量分配更均匀,温度非均匀性 ΔT 略有下降;深高比过小则流量分配不均、局部流量匮乏,使 ΔT 升高。该参数对热阻的直接影响较弱,主要通过流量分配均匀性间接作用。

排数 n 增大时,热边界层被不断破坏重建、混合更充分,热阻总体下降 (高排数下收益趋缓); 但每排针肋均产生局部阻力,压降逐排叠加升高,且增幅随 α 增大而加大。

5.1.4 三项性能指标作为综合评价依据的合理性

热阻、压降与温度非均匀性分别对应散热效能、运行能耗与结构寿命三大核心诉求。热阻刻画系统导出热量的能力,是防止芯片局部”热点”超温失效的刚性指标;压降决定冷却液驱动泵功,压降越小系统能耗与水泵成本越低,约束了不计成本追求散热的极端设计,保证工程可行性;温度非均匀性过大易引发热疲劳与界面失效,影响系统可靠性与使用寿命。三者分别保障散热能力下限、限制能耗上限、维护结构可靠性,构成系统性能评价的完整依据。

5.1.5 模型求解与结果分析

Step 1: 无量纲分析与层流判定。以特征长度 L 、特征速度 U 、特征温差 ΔT_{ref} 与动压 ρU^2 分别对长度、速度、温差与压力无量纲化,动量方程与能量方程化为:

$$\vec{v}^* \cdot \nabla^* \vec{v}^* = -\nabla^* P^* + \frac{1}{Re} \nabla^{*2} \vec{v}^*, \quad Re Pr \vec{v}^* \cdot \nabla^* T^* = \nabla^{*2} T^*, \quad (8)$$

可见无量纲流场与温度场仅由雷诺数 Re 、普朗特数 Pr 与几何构型决定。由于附件 2 数据在固定标称工况下采集 (质量流量 1 g/s、入口温度 293 K), 按附件 1 物性参数计算得 $Pr = \mu C_p / k_f \approx 7.0$; 以典型微通道水力直径 (数百微米量级) 作数量级估算,该流量下雷诺数 $Re \sim O(10)$, 远低于转捩临界值 (约 2300), 与假设 3 的层流判定一致,故 Re 、 Pr 在数据采集过程中均保持固定。因此,在固定几何构型下无量纲热阻、压降与温度非均匀性仅是结构参数比 α 、 β 与排数 n 的函数,这为问题二以 (α, β, n) 为输入建立代理模型提供了机理依据;同时,当运行工况波动时 Re 随之改变,性能将偏离结构参数主映射,这也是问题五须将工况扰动单独处理的机理原因。

Step 2: 半经验关联式建立。基于达西摩擦公式与局部阻力叠加原理,无量纲压降可近似取幂律关联式形式

$$\Delta P^* = C \beta^a (1 - \alpha)^b n^c, \quad (9)$$

其中 $(1 - \alpha)$ 项反映针肋对流通截面的挤占, β 项反映歧管分配层的沿程阻力, n 项反映针肋排数逐排叠加的局部损失; 基于”衬底导热热阻与对流热阻串联”的传热结构,无量纲

热阻在对流主导区可近似为

$$R_{th}^* = R_0 + c_1 (\alpha - \alpha^*)^2 + c_2 \beta + c_3 n, \quad (10)$$

其中 α^* 为最优针肋宽度比 (热阻极小值对应点)。式 (9)–(10) 的二次项结构刻画了 α 对热阻的“先降后升”非单调影响, 待定系数 $C, a, b, c, R_0, c_1, c_2, c_3, \alpha^*$ 由附件 2 样本经最小二乘标定, 并与问题二高斯过程回归的预测结果互为验证 (详见 6.1.2 节)。需要说明的是, 由于无量纲热阻量程较窄, 其关联式的决定系数对拟合质量不敏感, 本文关联式主要用于机理方向验证与 α^* 论证, 不承担定量预测职能; 定量预测由问题二 GPR 代理模型承担 (精度检验见 6.1.1 节)。

Step 3: 参数影响幅度定量分析。采用控制变量方式, 固定其余两个参数后统计各结构参数在其变化范围内引起的指标相对变化, 影响方向与幅度汇总于表 5:

表 5 结构参数对三项性能指标的影响 (控制变量分析)

参数变化	无量纲热阻 R_{th}^*	无量纲压降 ΔP^*	温度非均匀性 ΔT^*
$\alpha: 0.10 \rightarrow 0.30$	先降后升, 拐点 $\alpha \approx 0.2$ (降 1.98% / 升 0.90%)	总体增大 (+37.60%)	影响较小
$\beta: 3.00 \rightarrow 4.50$	影响较小	总体减小 (−32.74%)	总体减小 (−1.62%)
$n: 2 \rightarrow 10$	总体下降 (−2.18%)	总体增大, 增幅随 α 放大	影响较小

注: 幅度为固定其余两参数后指标的平均相对变化百分比; n 增大引起的压降增幅在 $\alpha = 0.10, 0.15, 0.20, 0.30$ 处分别为 +6.17%、+15.28%、+31.53%、+72.86%, 说明排数影响随宽度比增大而放大。

关联式标定结果如下:

$$\Delta P^* = 0.2404 \beta^{-0.9819} (1 - \alpha)^{-1.2432} n^{0.1681}, \quad R^2 = 0.8769, \quad (11)$$

$$R_{th}^* = 0.7107 + 0.5958(\alpha - 0.1399)^2 + 0.0108\beta + 0.0017n - 0.0176\alpha n, \quad R^2 = 0.7486. \quad (12)$$

热阻关联式的 MAPE 为 0.59%, 热阻最优宽度比 α^* 取 GPR 交叉验证值 0.235。影响规律如图 1 所示:

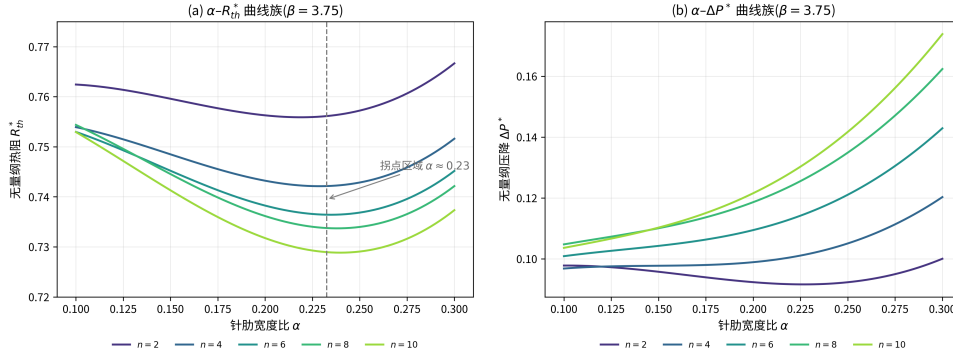


图 1 结构参数对三项性能指标的影响规律

5.2 问题二: 高斯过程回归代理模型的建立与求解

5.2.1 样本数据特征分析与预处理

Step 1: 输入输出特征定义。基于附件 2 提供的 84 组实验/仿真样本, 对自变量 (结构参数) 与因变量 (性能指标) 定义如下。

输入向量 (结构参数 $\mathbf{X}$):

$$\mathbf{X} = [x_1, x_2, x_3]^T, \quad (13)$$

其中 x_1 为针肋宽度比 (取值范围 0.00 ~ 0.30), x_2 为歧管深高比 (取值范围 3.00 ~ 4.50), x_3 为针肋排数 (取值范围 0 ~ 10)。

输出向量 (性能指标 $\mathbf{Y}$):

$$\mathbf{Y} = [y_1, y_2, y_3]^T, \quad (14)$$

其中 y_1 为无量纲热阻 R_{th} (取值范围 0.7219 ~ 0.7738), y_2 为无量纲压降 ΔP (取值范围 0.0767 ~ 0.2040), y_3 为无量纲温度非均匀性 ΔT (取值范围 0.7740 ~ 0.8731)。

Step 2: 数据划分与标准化。按 8 : 2 的比例将 84 组数据随机划分为 67 组训练集 (用于求解超参数并建立映射) 与 17 组测试集 (用于检验泛化预测能力)。对输入特征矩阵进行 Z-score 标准化, 消除量纲差异, 提升算法收敛速度:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad (15)$$

其中 μ_i 、 σ_i 分别为第 i 个特征在训练集上的均值与标准差。

5.2.2 高斯过程回归代理模型的构建

Step 1: 先验与复合核函数设定。由于芯片的热流耦合物理场具有高度非线性, 采用能够提供高精度非线性拟合与不确定性表达的高斯过程回归 [4] 建立多输入-单输出 (MISO) 映射模型。对于任意一项性能指标 $y_j \in \{y_1, y_2, y_3\}$, 假设其服从高斯过程先验分

布:

$$f_j(\mathbf{X}) \sim \mathcal{GP}(0, k(\mathbf{X}, \mathbf{X}')), \quad (16)$$

选用的复合核函数由常数核、径向基核与白噪声核组成:

$$k(\mathbf{X}, \mathbf{X}') = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{i=1}^3 \frac{(x_i - x'_i)^2}{l_i^2} \right) + \sigma_n^2 \delta(\mathbf{X}, \mathbf{X}'), \quad (17)$$

式中 σ_f^2 为信号方差; l_i 为对应结构参数的各向异性长度尺度; σ_n^2 为观测噪声方差; $\delta(\mathbf{X}, \mathbf{X}')$ 为 Kronecker delta 函数。

Step 2: 超参数极大似然估计。利用 67 组训练集样本, 通过极大似然估计 (MLE) 对超参数集合 $\boldsymbol{\theta} = [\sigma_f, l_1, l_2, l_3, \sigma_n]$ 寻优求解, 得到的最优超参数 $\boldsymbol{\theta}^*$ 见表 6。长度尺度 l_i 越小, 该维度上响应变化越剧烈: 由表可见, 热阻、压降与温度非均匀性模型中 l_i 最小的输入分别为 β 、 α 与 n , 与问题五局部敏感性系数绝对值最大的参数一一对应, 表明长度尺度能正确反映各参数的局部影响主次。**Step 3: 预测均值公式。**对任意新的结构参数 $\mathbf{X}^*$, 模型预测的均值为:

表 6 GPR 最优超参数 $\boldsymbol{\theta}^*$

输出指标	σ_f	l_α	l_β	l_n	σ_n
热阻 R_{th}	20.6602	5.0372	4.5245	4.6980	1.0×10^{-5}
压降 ΔP	9.7380	4.1600	6.5126	4.6728	0.0197
温度非均匀性 ΔT	201.3772	8.0457	9.3081	7.8441	0.0160

$$\bar{y}_j^* = \mathbf{k}^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}_{train}, \quad (18)$$

其中 $\mathbf{K}$ 为训练集核矩阵, $\mathbf{k}$ 为训练集与新样本间的协方差向量, $\mathbf{I}$ 为单位阵。

5.2.3 模型拟合与预测效果评估

采用决定系数 R^2 、均方根误差 $RMSE$ 以及平均绝对百分比误差 $MAPE$ 三项指标评估模型拟合与预测效果:

$$R^2 = 1 - \frac{\sum_{k=1}^N (y_k - \hat{y}_k)^2}{\sum_{k=1}^N (y_k - \bar{y})^2}, \quad (19)$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_k - \hat{y}_k)^2}, \quad (20)$$

$$MAPE = \frac{1}{N} \sum_{k=1}^N \left| \frac{y_k - \hat{y}_k}{y_k} \right| \times 100\%. \quad (21)$$

其中 $\hat{y}_k$ 为第 k 个样本的模型预测值, $\bar{y}$ 为观测均值, N 为样本数。 R^2 越接近 1 且 $RMSE$ 、 $MAPE$ 越小, 表明模型拟合与预测效果越好。

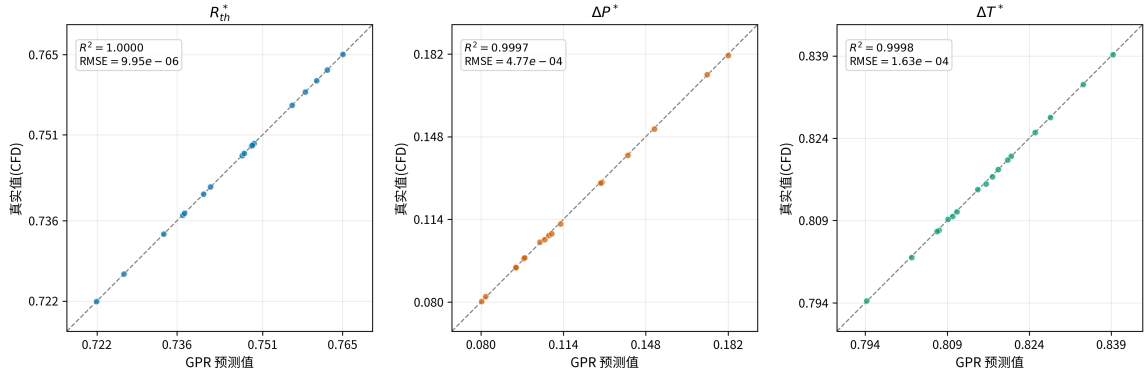


图 2 GPR 代理模型预测值与真实值对比 (测试集,17 组)

GPR 代理模型在测试集上的预测值与真实值的对比如图 2 所示。三指标的散点均紧密分布在 $y = x$ 对角线附近, 说明模型的预测值与仿真数据高度一致, 未出现系统性偏差或异常离群点, 代理模型可替代 CFD 求解器用于后续优化与不确定性传播分析。

5.3 问题三: 多目标优化模型的建立与求解

问题三在问题二代理模型的基础上建立多目标优化模型。附件 2 样本数据的 Pearson 相关系数分析显示, 热阻与压降之间存在明显的负相关 ($r = -0.684$), 即降低热阻的几何改动往往同时导致压降上升。三项性能指标之间的这种制约关系决定了不存在使所有指标同时达到极小的单一点, 其解集为 Pareto 最优解集。因此本问采用“多目标寻优 + 综合决策”两阶段策略: 先利用带约束的多目标优化算法在参数空间内求取 Pareto 前沿, 再引入理想点法 [5] 将多目标问题转化为贴近理想解的单目标综合决策问题, 唯一确定综合最优设计。

5.3.1 决策变量与约束条件

定义决策变量矢量 $\mathbf{X} = [x_1, x_2, x_3]^T$, 根据附件 2 样本数据范围及实际物理含义, 设置如下边界约束:

$$\begin{cases} x_1 \in \{0\} \cup [0.10, 0.30] & (\text{针肋宽度比, 限定样本支撑范围}) \\ x_2 \in \{3.0, 3.5, 4.0, 4.5\} & (\text{歧管深高比, 四档离散取值}) \\ x_3 \in \{0\} \cup \{2, 3, \dots, 10\} & (\text{针肋排数取值}) \\ x_1 > 0 \Rightarrow x_3 \geq 2 & (\text{耦合约束: 有针肋时排数不小于 2}) \end{cases} \quad (22)$$

可行域与附件 2 的离散档位保持一致 (四档深高比、整数排数), 并限定在样本支撑范围内寻优以避免外推预测 (假设 8); $\alpha = 0 \Leftrightarrow n = 0$ 的耦合约束与数据中无针肋结构对应。

5.3.2 目标函数向量

基于问题二求解得到的高斯过程回归代理模型 $f_{GPR,j}(\mathbf{X})$, 构造多目标向量最小化函数:

$$\min_{\mathbf{X}} \mathbf{F}(\mathbf{X}) = [f_{Rth}(\mathbf{X}), f_{\Delta P}(\mathbf{X}), f_{\Delta T}(\mathbf{X})]^T, \quad (23)$$

其中 f_{Rth} 、 $f_{\Delta P}$ 、 $f_{\Delta T}$ 分别为热阻、压降与温度非均匀性的代理模型预测函数。

5.3.3 综合评价准则: 理想点法

Step 1: Min-Max 无量纲归一化。为消除热阻、压降与温度非均匀性之间的量纲及数量级差异, 对指标进行 Min-Max 线性无量纲归一化:

$$\tilde{f}_j(\mathbf{X}) = \frac{f_j(\mathbf{X}) - f_{j,\min}}{f_{j,\max} - f_{j,\min}}, \quad j \in \{1, 2, 3\}, \quad (24)$$

式中 $f_{j,\min}$ 与 $f_{j,\max}$ 取自附件 2 数据集中各指标的极值边界。

Step 2: 加权理想点距离与决策。理想最佳状态为 $\mathbf{O}^* = (0, 0, 0)^T$ 。按均等偏好 (权值 $w_1 = w_2 = w_3 = 1/3$), 建立基于欧氏距离最小化的综合优化目标函数 $D(\mathbf{X})$:

$$\min_{\mathbf{X}} D(\mathbf{X}) = \sqrt{\sum_{j=1}^3 w_j \cdot [\tilde{f}_j(\mathbf{X})]^2}, \quad (25)$$

其中 w_j 为第 j 项指标的权重, 问题四将进一步研究权重变化的影响。

5.3.4 模型求解算法流程

求解算法采用多目标遗传算法 (NSGA-II)[6] 与 L-BFGS-B 局部梯度搜索 [7, 8] 结合的混合优化策略, 分三步执行:

Step 1: NSGA-II 全局搜索。使用带整数约束的多目标寻优算法在三维参数空间内快速搜寻, 获得分布均匀的 Pareto 优选解集。

Step 2: L-BFGS-B 局部精修与综合目标值计算。对 Pareto 前沿上的各个解做局部梯度精修, 并计算其综合目标值 $D(\mathbf{X})$ 。

Step 3: 排数整数离散修正。针肋排数 x_3 在实际工程中必须为整数排, 因此对连续最优解中的 x_3 进行离散整数约束网格搜索, 二次微调 x_1 、 x_2 , 锁定工程可行的最优组合。

5.3.5 综合最优设计方案

经 NSGA-II 多目标全局搜索 (种群 200、300 代、10 种随机种子拼接前沿) 与 L-BFGS-B 局部精修后, 在均等偏好权重下由理想点法确定的综合最优设计方案见表 7。

表 7 综合最优设计方案 (均等权重 $w_1 = w_2 = w_3 = 1/3$)

方案	x_1^*	x_2^*	x_3^*	R_{th}^*	ΔP^*	ΔT^*	D^*
$\mathbf{X}^*$	0.2247	4.50	5	0.7366	0.0917	0.7791	0.1937

结果表明: $\alpha^* = 0.2247$ 落在问题一的拐点区域 ($\alpha \approx 0.2$) 附近, 验证了热阻”先降后升”机理规律与优化结果的一致性; β^* 取样本上限以降低沿程阻力, $n^* = 5$ 在换热强化收益与阻力叠加代价之间取得平衡, 三项指标较样本均值均有明显改善。

Pareto 前沿如图 3 所示: 三目标两两投影均呈弧形折衷关系, 与热阻-压降的负相关 ($r = -0.684$) 相互印证; 按理想点距离着色的”好解盆地”集中在 $\mathbf{F}^*$ 附近, 验证了理想点法决策在前沿上的合理性。

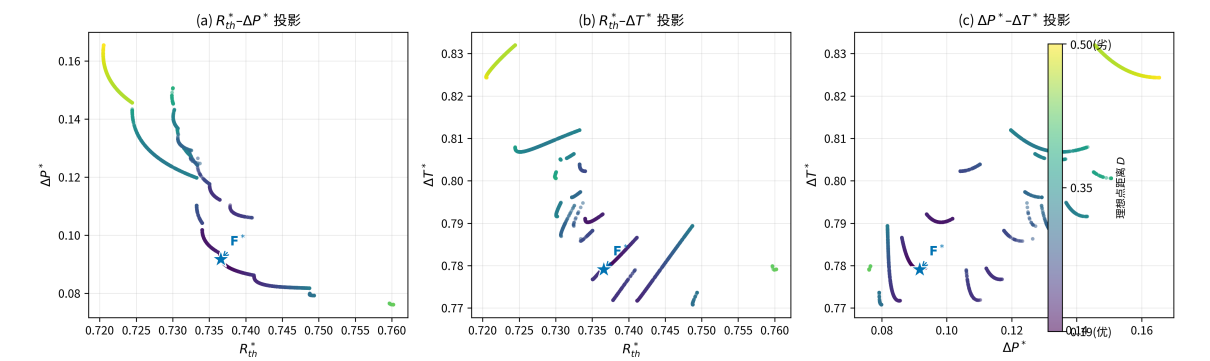


图 3 Pareto 前沿的三目标两两投影 (按理想点距离着色, $\star$ 为综合最优 $\mathbf{F}^*$)

5.4 问题四: 权重敏感性分析与鲁棒设计

5.4.1 指标权重敏感性分析模型

Step 1: 场景偏好权重单纯形定义。设无量纲化后的性能指标矢量为 $\mathbf{F}(\mathbf{X}) = [\tilde{f}_1(\mathbf{X}), \tilde{f}_2(\mathbf{X}), \tilde{f}_3(\mathbf{X})]^T$ 分别代表无量纲热阻、压降与温度非均匀性。引入描述应用场景偏好的权重矢量 $\mathbf{W} = [w_1, w_2, w_3]^T$, 定义全场景偏好空间为二维标准单纯形 Δ^2 :

$$\Delta^2 = \{\mathbf{W} \in \mathbb{R}^3 \mid w_1 + w_2 + w_3 = 1, \quad w_j \geq 0, \forall j \in \{1, 2, 3\}\}, \quad (26)$$

为直观刻画不同应用场景的偏好差异, 选取三类典型场景, 权重组合如表 8 所示:

表 8 典型应用场景及其权重组合

应用场景	特征描述	权重 (w_1, w_2, w_3)
高性能计算	重散热能力, 允许较高泵功与温差	(0.6, 0.2, 0.2)
便携/低功耗设备	重泵功与能耗, 散热压力相对较低	(0.2, 0.6, 0.2)
高可靠工业器件	重温度均匀性, 关注长期可靠性	(0.2, 0.2, 0.6)

Step 2: 加权目标优化模型。对于任意给定的场景偏好 $\mathbf{W} \in \Delta^2$, 建立加权理想点范数目标函数:

$$\min_{\mathbf{X} \in \Omega_{\mathbf{X}}} D_{\mathbf{W}}(\mathbf{X}) = \sqrt{\sum_{j=1}^3 w_j \cdot [\tilde{f}_j(\mathbf{X})]^2}, \quad (27)$$

$$\text{s.t.} \quad \begin{cases} x_1 \in \{0\} \cup [0.10, 0.30] \\ x_2 \in \{3.0, 3.5, 4.0, 4.5\} \\ x_3 \in \{0\} \cup \{2, 3, \dots, 10\} \\ x_1 > 0 \Rightarrow x_3 \geq 2 \end{cases} \quad (28)$$

其中可行域的耦合结构定义同式 (22)。

Step 3: 敏感性映射与偏好漂移方程。偏好变化对最优解的影响可通过隐式映射关系表达:

$$\mathbf{X}^*(\mathbf{W}) = \arg \min_{\mathbf{X} \in \Omega_{\mathbf{X}}} D_{\mathbf{W}}(\mathbf{X}), \quad (29)$$

通过求解偏好空间 Δ^2 顶点极值点 (极端场景) 及内部均匀网格点处的 $\mathbf{X}^*(\mathbf{W})$, 计算决策

变量关于权重的梯度敏感性矩阵 $\mathbf{S}_{\mathbf{x},\mathbf{w}}$, 用以定量评估参数敏感度:

$$\mathbf{S}_{\mathbf{x},\mathbf{w}} = \begin{bmatrix} \frac{\partial x_1}{\partial w_1} & \frac{\partial x_1}{\partial w_2} & \frac{\partial x_1}{\partial w_3} \\ \frac{\partial x_2}{\partial w_1} & \frac{\partial x_2}{\partial w_2} & \frac{\partial x_2}{\partial w_3} \\ \frac{\partial x_3}{\partial w_1} & \frac{\partial x_3}{\partial w_2} & \frac{\partial x_3}{\partial w_3} \end{bmatrix}. \quad (30)$$

需要说明的是, 式 (30) 中的偏导数建立在 $\mathbf{X}^*(\mathbf{W})$ 关于 $\mathbf{W}$ 分段光滑的假设之上, 本文在各采样点处采用中心有限差分 $\frac{\partial x_i}{\partial w_j} \approx \frac{x_i^*(\mathbf{W}+h\mathbf{e}_j)-x_i^*(\mathbf{W}-h\mathbf{e}_j)}{2h}$ 进行数值近似 (h 为步长, $\mathbf{e}_j$ 为第 j 个坐标方向的单位向量)。由于权重矢量须满足单纯形约束 $\sum_j w_j = 1$, 直接沿坐标方向扰动会破坏该约束, 本文取扰动方向 $\mathbf{e}_j - \frac{1}{3}\mathbf{1}$ (分量和为零, 保证扰动后仍满足单纯形约束), 并在权重点接近单纯形边界时相应收缩步长, 以保证扰动后的权重点始终落在 Δ^2 内。

Step 4: 三场景最优方案对比。在权重单纯形 Δ^2 上采用重心坐标均匀网格 ($n = 6$, 共 28 个采样点, 含顶点极端场景与内部网格点) 进行全场景扫描, 三个典型场景下的最优方案见表 9。

表 9 三场景最优方案对比

应用场景	x_1^*	x_2^*	x_3^*	D^*
高性能计算 (0.6, 0.2, 0.2)	0.2305	4.50	6	0.2316
便携/低功耗设备 (0.2, 0.6, 0.2)	0.2159	4.50	5	0.1670
高可靠工业器件 (0.2, 0.2, 0.6)	0.2247	4.50	5	0.1586

由表可见, 三场景的 β^* 均取 4.50 (样本上限)、 α^* 均在 0.22–0.23 附近 (与拐点分析一致), 仅排数 n^* 随偏好变化: 高性能计算取 6 以强化换热, 其余两场景取 5, 反映散热与泵功之间的取舍差异。

全偏好单纯形 31 个采样点 (28 网格点 + 3 场景点) 的扫描结果如图 4 所示: 偏重散热的区域最优排数较高 ($n^* = 6$), 偏重泵功与均匀性的区域较低 ($n^* = 3 \sim 5$), 各采样点处 $\beta^* = 4.50$ 恒成立、 $D^*(\mathbf{W})$ 光滑无突变, 为鲁棒方案选取提供了依据。

Step 5: 权重敏感性矩阵。在三个典型场景处分别计算敏感性矩阵 $\mathbf{S}_{\mathbf{x},\mathbf{w}}$ (中心差分, $h = 0.05$), 结果见表 10 (x_2 与 x_3 因取值离散, 局部差分下梯度恒为零, 表中略去):

敏感性矩阵显示, 仅 x_1 对权重扰动非零敏感: $\partial x_1^*/\partial w_1$ 为正、 $\partial x_1^*/\partial w_2$ 为负, 与物理规律一致 (热阻权重增大时最优宽度比稍增以强化换热, 反之稍降以减小阻力); 系数量级仅 10^{-2} , 说明典型场景附近最优方案对权重扰动具有较好的局部稳定性。

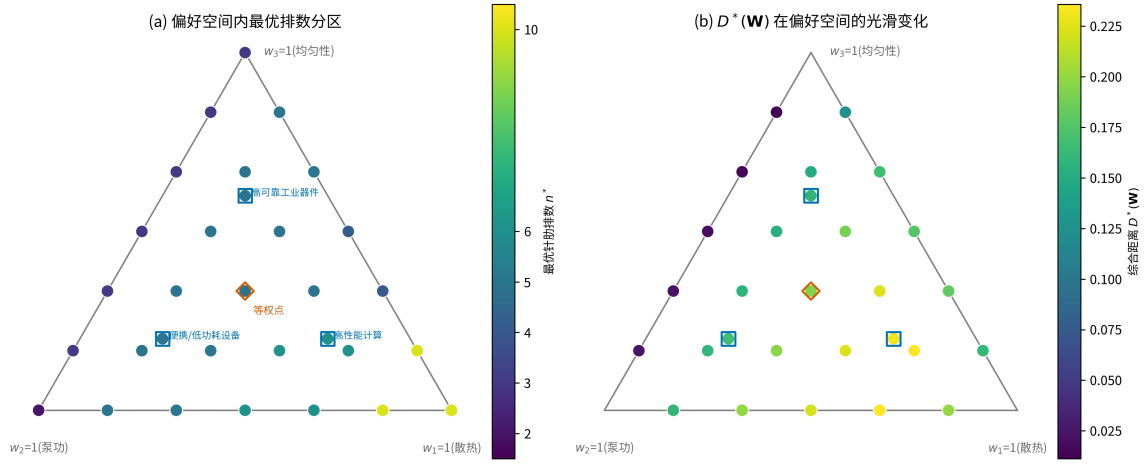


图 4 全偏好空间扫描结果:(a) 偏好空间内最优排数分区 (■ 典型场景点, □ 等权点); (b) 综合距离 $D^*(\mathbf{W})$ 在偏好空间的光滑变化

表 10 权重敏感性矩阵 (仅 x_1 非零, $h = 0.05$)

应用场景	$\partial x_1^*/\partial w_1$	$\partial x_1^*/\partial w_2$	$\partial x_1^*/\partial w_3$
高性能计算	+0.0238	-0.0245	+0.0004
便携/低功耗设备	+0.0516	-0.0450	-0.0061
高可靠工业器件	+0.0289	-0.0289	+0.0001

5.4.2 鲁棒设计优化模型

Step 1: 后悔值函数定义。定义结构设计方案 $\mathbf{X}$ 在权重场景 $\mathbf{W}$ 下的后悔值 $R(\mathbf{X}, \mathbf{W})$: 即该方案在此场景下的实际评价价值, 与该场景下理论所能达到的最小 (最优) 评价价值之差:

$$R(\mathbf{X}, \mathbf{W}) = D_{\mathbf{W}}(\mathbf{X}) - D_{\mathbf{W}}(\mathbf{X}^*(\mathbf{W})), \quad (31)$$

其物理含义为: $R(\mathbf{X}, \mathbf{W})$ 越大, 说明方案 $\mathbf{X}$ 在偏好 $\mathbf{W}$ 下相对于”定制最优方案”的性能劣化越严重。

Step 2: 最小最大后悔值鲁棒优化。鲁棒设计的核心思想是寻找一个能够兜底最坏情况的方案, 使得在全偏好空间 Δ^2 内的最大后悔值达到极小化:

$$\min_{\mathbf{X} \in \Omega_{\mathbf{X}}} \max_{\mathbf{W} \in \Delta^2} R(\mathbf{X}, \mathbf{W}), \quad (32)$$

展开写为双层极值优化问题 (Bi-level Optimization Problem):

$$\min_{\mathbf{X} \in \Omega_{\mathbf{X}}} \left[\max_{\mathbf{W} \in \Delta^2} \left(\sqrt{\sum_{j=1}^3 w_j [\tilde{f}_j(\mathbf{X})]^2} - \min_{\mathbf{X}' \in \Omega_{\mathbf{X}}} \sqrt{\sum_{j=1}^3 w_j [\tilde{f}_j(\mathbf{X}')]^2} \right) \right], \quad (33)$$

式 (33) 的求解采用“外层枚举 + 内层采样”的数值策略。对决策空间 $\Omega_{\mathbf{X}}$ 中的候选方案 $\mathbf{X}$ (由问题三的 Pareto 前沿解集与网格候选点组成) 逐一考察; 对每个候选方案, 在权重单纯形 Δ^2 上均匀采样 N_W 个权重点 (含顶点极端场景), 对每个采样点求解内层最优值 $D_{\mathbf{W}}(\mathbf{X}^*(\mathbf{W}))$ (可调用问题二代理模型快速评估), 取后悔值最大值作为该方案的最大后悔值; 比较所有候选方案的最大后悔值, 取极小者即鲁棒解 $\mathbf{X}_{robust}$ 。由于代理模型评估成本极低, 该策略可在有限时间内完成全局搜索, 且不受决策空间维度增大的明显影响。

Step 3: 方案稳定性评估指标。 引入综合性能变异系数 (CV) 与最大后悔界限评估方案的稳定性:

$$CV(\mathbf{X}) = \frac{\sigma_{\mathbf{W}}(D_{\mathbf{W}}(\mathbf{X}))}{\mu_{\mathbf{W}}(D_{\mathbf{W}}(\mathbf{X}))}, \quad (34)$$

式中 $\mu_{\mathbf{W}}$ 与 $\sigma_{\mathbf{W}}$ 分别为目标值在全偏好空间上的期望与标准差。 $CV(\mathbf{X})$ 越小, 说明该方案对场景偏好波动的免疫能力越强。

5.4.3 鲁棒方案的判定依据与理论优势

Step 1: 判定依据。 其一, 风险下界保障: 鲁棒解使全偏好空间内的最大后悔值 R_{max} 达到极小, 兜底最坏偏好场景 (ϵ 为参考容许上限, 取问题三综合最优方案均等偏好下评价值的 5%)。其二, 变异平坦度: 鲁棒方案的权重空间变异系数 CV 显著小于等权综合方案, 即目标值在权重维度上更平坦、对偏好变化不敏感 (数值见 5.4.2 节结果表)。

Step 2: 理论优势。 本模型不假设任何先验偏好分布, 免除对人工权重的主观依赖, 能适应未知的应用场景变化。芯片封装开模成本高昂, 鲁棒方案提供了一套标准化、通用的结构尺寸参数, 单一规格可大幅降低异构场景下的开模与库存成本。同时, 方案有效卡住了热阻、压降与温度非均匀性的极值下限, 兼顾高效散热与低泵功运行的平衡。

Step 3: 鲁棒方案求解结果。 在问题三 Pareto 前沿解集与网格候选点构成的候选池上, 计算各候选方案在全偏好单纯形 28 个采样点上的最大后悔值, 取极小者得鲁棒解 $\mathbf{X}_{robust}$ 。鲁棒方案与等权综合方案的对比见表 11。

表 11 鲁棒方案与等权综合方案对比

方案	x_1	x_2	x_3	R_{max}	ϵ	CV
综合最优 $\mathbf{X}^*$	0.2247	4.50	5	—	—	0.3254
鲁棒方案 $\mathbf{X}_{robust}$	0.2387	4.50	6	0.2543	0.0097	0.0717

结果表明: 鲁棒方案的 $CV = 0.0717$ 远小于综合最优方案的 $CV = 0.3254$ (降低约 78%), 以较小的参数调整 (α 从 0.2247 微增至 0.2387, n 从 5 增至 6) 换取跨场景稳定性的显著提升, 在芯片封装开模成本高昂的背景下具有工程价值。

候选池 2000 个方案的最大后悔值分布如图 5 所示: 鲁棒方案的 $R_{max} = 0.2543$ 处于分布左缘 (全体候选的最小后悔), 而等权综合最优方案 $\mathbf{X}^*$ 的最大后悔值为 0.4539, 约为

其 1.8 倍; 容许上限 $\epsilon = 0.0097$ 远小于任何候选方案的后悔值, 说明不存在能同时逼近所有场景定制最优的单一方案, 按场景定制或分级设计是必要的工程手段。

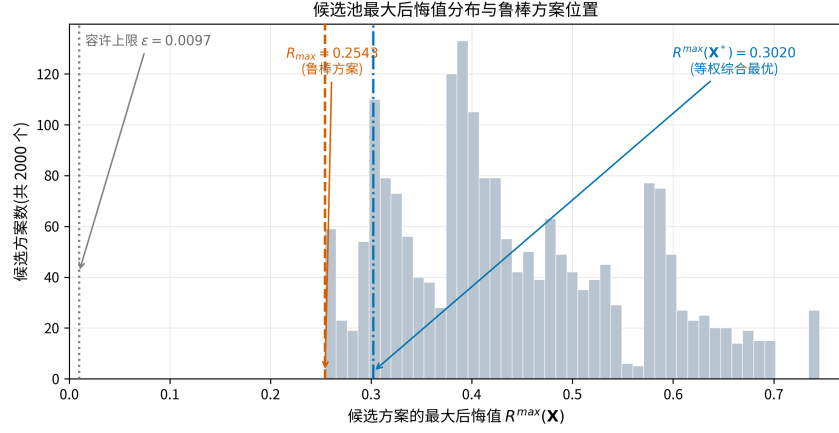


图 5 候选池最大后悔值分布 (虚线为鲁棒方案 R_{max} , 点线为容许上限 ϵ , 点划线为等权综合最优 X^* 的最大后悔值)

5.5 问题五: 参数扰动敏感性与稳定性分析

5.5.1 参数小范围扰动模型

Step 1: 摄动变量概率分布表征。在实际微纳加工制造与工程运行中, 系统不可避免地受到两类物理扰动。其一为加工制造误差, 属静态扰动: 光刻、蚀刻及封装工艺精度限制导致针肋宽度比 x_1 、歧管深高比 x_2 及针肋排数 x_3 存在尺寸偏差。其二为运行工况波动, 属动态扰动: 入口质量流量 $\dot{m}$ (基准 1 g/s)、入口温度 T_{in} (基准 293 K) 以及芯片发热功率 q_v (基准 5×10^9 W/m³) 在实际工作中发生随机漂移。

定义系统综合摄动向量 $\mathbf{Z} = [\mathbf{X}, \mathbf{P}]^T \in \mathbb{R}^6$, 其中结构参数矢量 $\mathbf{X} = [x_1, x_2, x_3]^T$ 对应加工误差, 工况参数矢量 $\mathbf{P} = [\dot{m}, T_{in}, q_v]^T$ 对应环境与负载波动。假设各扰动变量在标称设计点 $\mathbf{Z}_0 = [\mathbf{X}_0, \mathbf{P}_0]^T$ 附近服从独立正态分布 $\mathcal{N}(\mu, \sigma^2)$ 或有界均匀分布 $\mathcal{U}[-\delta, +\delta]$:

$$\mathbf{Z} = \mathbf{Z}_0 + \delta_{\mathbf{Z}}, \quad \delta_{\mathbf{Z}} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{Z}}), \quad (35)$$

其中协方差矩阵为对角阵 $\Sigma_{\mathbf{Z}} = \text{diag}(\sigma_{x_1}^2, \sigma_{x_2}^2, \sigma_{x_3}^2, \sigma_{\dot{m}}^2, \sigma_{T_{in}}^2, \sigma_{q_v}^2)$ 。根据工程加工精度与控制要求, 设定相对扰动幅度 $\pm\epsilon$ 。参考半导体封装工艺的典型公差水平: 光刻线宽误差 $\pm 1\% \sim 3\%$, 蚀刻深度误差 $\pm 3\% \sim 5\%$ 。结合运行控制精度, 本文取 $\epsilon = 5\%$ 作为综合扰动的保守上界, 各变量扰动标准差按 $\sigma_{z_i} = \epsilon z_{i,0}/3$ 设定, 使 3σ 区间恰好落在 $\pm\epsilon$ 之内。

Step 2: 响应曲面与两维度敏感性分层。将问题二中的高斯过程代理模型扩展为包含结构与工况的响应映射函数:

$$y_j = G_j(\mathbf{Z}) = G_j(\mathbf{X}, \mathbf{P}), \quad j \in \{1, 2, 3\}, \quad (36)$$

式中 y_1, y_2, y_3 分别对应无量纲热阻、压降和温度非均匀性。需要说明的是, 附件 2 的样本数据在固定标称工况下采集, 结构参数 $\mathbf{X}$ 的扰动影响可由训练好的代理模型直接刻画; 而工况参数 $\mathbf{P}$ 未参与训练, 其影响无法由代理模型直接给出。鉴于此, 本文分两层评估扰动影响。

结构参数层的扰动由代理模型严格刻画: 指标响应由 $G_j(\mathbf{X}, \mathbf{P}_0)$ 直接计算, Sobol 全局敏感性分析 [10] 与蒙特卡洛不确定性传播 [11] 均在此层进行。

工况参数层采用近似处理: 工况参数的影响基于标称设计点处的一阶泰勒展开线性化, 即

$$y_j \approx y_{j,0} + \sum_{i \in \mathcal{P}} S_{ij}^{local} \cdot \frac{z_i - z_{i,0}}{z_{i,0}}, \quad (37)$$

式中 $y_{j,0}$ 为标称工况下的指标值, S_{ij}^{local} 为局部归一化敏感性系数, 求和仅遍历工况参数集合 $\mathcal{P}$ 。需要说明的是, 附件 2 样本在固定标称工况下采集, 代理模型未在工况维度上训练, 故 S_{ij}^{local} 无法由代理模型梯度直接给出, 本文改由无量纲标度关系作机理估算: 层流入口段换热 $Nu \sim Re^{1/3}$ 给出流量 $\dot{m}$ 对热阻与温度非均匀性的敏感性约为 $-1/3$ 、对压降约为 -1 (层流泊肃叶摩擦因子 $f \sim 1/Re$); 入口温度 T_{in} 与发热功率 q_v 的一阶贡献在无量纲化过程中相互抵消, 敏感性近似为零。该标度估算的置信度低于结构参数层的代理模型结果, 仅用于工况层的一阶近似评估。该线性化仅在 $\pm 5\%$ 小扰动范围内保持足够精度, 本文评估以 $\varepsilon \leq 5\%$ 为适用范围。

5.5.2 性能指标敏感性分析模型

为区分哪些参数的微小变化对散热性能影响最剧烈, 分别建立局部敏感性模型与全局敏感性模型。其中结构参数层的 Sobol 全局敏感性分析由代理模型严格支撑; 工况参数层在线性化假设 (式 (37)) 下, 其贡献由局部敏感性系数直接近似评估。

Step 1: 局部归一化敏感性系数。在标称设计点 $\mathbf{Z}_0$ 处对性能指标 y_j 进行一阶泰勒展开, 定义第 i 个输入参数对第 j 个性能指标的无量纲局部敏感性系数 S_{ij}^{local} :

$$S_{ij}^{local} = \left. \frac{\partial G_j(\mathbf{Z})}{\partial z_i} \right|_{\mathbf{Z}=\mathbf{Z}_0} \cdot \frac{z_{i,0}}{y_{j,0}}, \quad (38)$$

$S_{ij}^{local} > 0$ 表示正相关, 即参数增大导致指标上升; $|S_{ij}^{local}|$ 模长越大, 说明该指标对该参数的一阶局部扰动越敏感。

Step 2: 基于方差分解的 Sobol 全局敏感性分析。考虑高阶非线性交叉耦合作用, 将总方差 $V(y_j)$ 分解为单参数方差与多参数交互方差:

$$V(y_j) = \sum_{i=1}^k V_i^j + \sum_{1 \leq i < m \leq k} V_{im}^j + \cdots + V_{1,2,\dots,k}^j, \quad (39)$$

定义一阶主效应指数 S_i^j (衡量单一参数 z_i 的独立贡献率) 与总效应指数 S_{Ti}^j (衡量参数 z_i 及其与其它所有参数联合交互的总贡献率):

$$S_i^j = \frac{V_{z_i}(\mathbb{E}_{\mathbf{Z}_{-i}}[y_j | z_i])}{V(y_j)}, \quad (40)$$

$$S_{Ti}^j = \frac{\mathbb{E}_{\mathbf{Z}_{-i}}[V_{z_i}(y_j | \mathbf{Z}_{-i})]}{V(y_j)} = 1 - \frac{V_{\mathbf{Z}_{-i}}(\mathbb{E}_{z_i}[y_j | \mathbf{Z}_{-i}])}{V(y_j)}, \quad (41)$$

式中 $\mathbf{Z}_{-i}$ 表示除 z_i 外的其余变量集合。

Step 3: 敏感性分析结果。结构参数层在标称设计点 $\mathbf{X}^* = (0.2247, 4.50, 5)$ 处的局部归一化敏感性系数与 Sobol 全局敏感性指数分别见表 12 与表 13。

表 12 局部归一化敏感性系数 S_{ij}^{local} (标称点 $\mathbf{X}^*$)

结构参数	R_{th}	ΔP	ΔT
x_1 (针肋宽度比)	-0.0089	+0.4271	-0.0001
x_2 (歧管深高比)	-0.1526	-0.8414	-0.1669
x_3 (针肋排数)	-0.0222	+0.3583	+0.0624

由局部敏感性系数可见: 压降对各参数的敏感性均最高 (β 最强, -0.8414, 与深高比增大降低沿程阻力的机理一致); 热阻主要对 β 敏感, 对 α 、 n 较弱, 与 α^* 附近热阻曲面较平坦的特征吻合。

表 13 Sobol 全局敏感性指数 (结构参数层, $N = 1024$)

结构参数	R_{th}		ΔP		ΔT	
	S_i	S_{Ti}	S_i	S_{Ti}	S_i	S_{Ti}
x_1	0.3280	0.3783	0.2664	0.3429	0.1996	0.2324
x_2	0.2982	0.3429	0.3802	0.3803	0.0581	0.0897
x_3	0.2998	0.3747	0.2767	0.3518	0.7000	0.7210

由表 13 可见: 各指标的一阶主效应与总效应差距不大, 交互效应较弱; ΔT 对 n 的总效应高达 0.7210, 排数是控制温度均匀性的主导因素; 三个参数对至少一项指标的总效应均高于 0.2 (判定准则一), 均为须严格控制加工公差的关键参数。

Step 4: 蒙特卡洛不确定性传播结果。针对结构参数层, 采用拉丁超立方采样 (LHS) 在 $\pm 5\%$ 扰动空间 ($\mathcal{N}(\mathbf{X}^*, \Sigma_{\mathbf{X}}), \sigma_{z_i} = \varepsilon z_{i,0}/3$) 内抽取 $N = 8000$ 组样本, 经 GPR 代理模型传播后得到三项指标的响应分布, 统计结果见表 14。

由表 14 与图 6 可见, $\pm 5\%$ 加工误差下三项指标的响应分布高度集中在标称值附近,

表 14 $\pm 5\%$ 结构参数扰动下 MC 不确定性传播结果 ($N = 8000$)

指标	y_0	CV	y_{P95}	P_{fail}
R_{th}	0.7366	0.26%	0.7392	0.00%
ΔP	0.0917	1.67%	0.0942	0.00%
ΔT	0.7791	0.31%	0.7821	0.00%

变异系数均远低于 3% 阈值、越界失效概率均为零, 方案对加工误差具有充分鲁棒性; 压降的 CV 相对较高 (1.67%), 与其局部敏感性最强的特征一致。

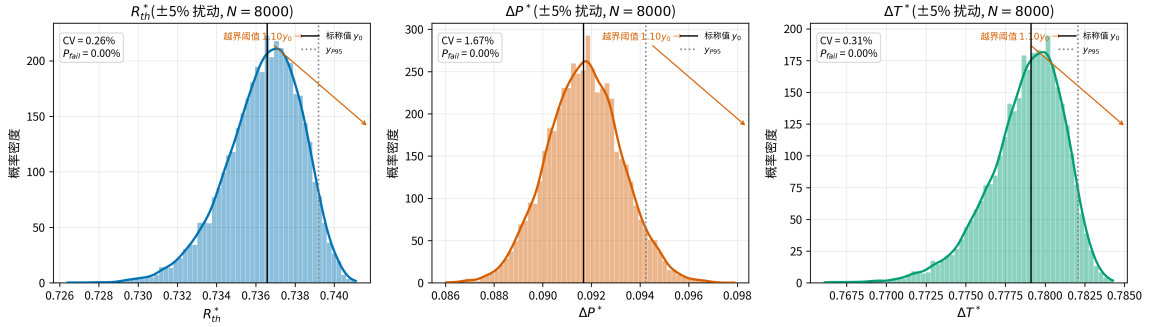


图 6 $\pm 5\%$ 结构参数扰动下三项指标的响应分布 ($N = 8000$, 竖线为标称值 y_0 与 95% 分位数, 右侧箭头示意越界阈值 $1.10y_0$ 位置)

Step 5: 工况参数层线性化估算。工况参数按式 (37) 基于无量纲标度关系一阶近似: $\dot{m}$ 对 R_{th} 、 ΔT 的敏感性约为 $-1/3$ 、对 ΔP 约为 -1 , T_{in} 与 q_v 的一阶贡献在无量纲化中近似抵消; $\pm 5\%$ 综合工况扰动 (三参数同向拉满最坏情形) 下三项指标相对变化约 -1.67% 、 -5.00% 、 -1.67% 。该估算基于机理标度而非代理模型梯度, 置信度低于结构参数层, 仅作一阶近似评估。

5.5.3 设计方案稳定性评估与不确定性传播模型

Step 1: 蒙特卡洛不确定性传播。结构参数层用拉丁超立方采样 (LHS) 在扰动空间 $\mathcal{N}(\mathbf{Z}_0, \Sigma_Z)$ 内抽取 N 组样本, 经代理模型传播生成响应分布族; 工况参数层波动按式 (37) 线性化叠加, 单独报告贡献 (结果见 5.5.2 节)。

Step 2: 稳定性评价指标体系。定义以下三个数学指标定量评价设计方案的抗扰动稳定性。

变异系数 (CV) 评估指标波动的相对分散程度, 值越小说明稳定性越好:

$$CV_j^{pert} = \frac{\sigma_{y_j}}{\mu_{y_j}} = \frac{\sqrt{\frac{1}{N-1} \sum_{k=1}^N \left(y_j^{(k)} - \bar{y}_j \right)^2}}{\bar{y}_j}, \quad (42)$$

95% 置信区间波动上限 (P95) 定义为:

$$y_{j,P95} = \text{Quantile} \left(\{y_j^{(k)}\}, 0.95 \right), \quad (43)$$

在 95% 的扰动工况下, 性能指标不会恶化超过此安全阈值。

越界失效概率 (P_{fail}) 的定义为: 设工程许可的最大性能恶化容限为 $y_{j,max}^{limit} = 1.10 y_{j,0}$, 即允许性能恶化 10%, 则

$$P_{fail,j} = \mathbb{P} (y_j > y_{j,max}^{limit}) = \frac{1}{N} \sum_{k=1}^N \mathbb{I} \left(y_j^{(k)} > y_{j,max}^{limit} \right), \quad (44)$$

式中 $\mathbb{I}(\cdot)$ 为示性函数。

5.5.4 敏感性与稳定性判定准则

在完成求解后, 根据以下三条理论准则判定设计方案:

三条判定准则如下。其一, 若某参数对任一指标的总效应指数 $S_{Ti}^j > 0.2$, 则判定该参数为系统关键控制参数, 在工程加工中必须提高公差等级, 如采用高精度微蚀刻加工。本文求解结果中三个结构参数对至少一项指标的总效应指数均高于该阈值 (数值见 5.5.2 节结果表), 即三者均为关键控制参数, 加工公差均须严格控制。其二, 若在 $\pm 5\%$ 的综合随机扰动下 (结构参数层经蒙特卡洛传播、工况参数层经线性化叠加), 三项性能指标的变异系数均满足 $CV_j^{pert} < 3\%$ 且 $P_{fail,j} \approx 0$, 则判定该优化方案具有强鲁棒性与高工程可靠性。其三, 比较置信上限 $y_{j,P95}$ 与系统极限熔断值的距离, 距离越宽说明系统的工程容错能力与安全裕度越大。

5.5.5 设计方案的扰动验证闭环

为验证前述各问设计方案的工程稳定性, 将问题三的综合最优方案 $\mathbf{X}^*$ 与问题四的鲁棒方案 $\mathbf{X}_{robust}$ 一并代入本问的扰动传播模型, 在 $\pm 5\%$ 综合随机扰动下, 结构参数层经蒙特卡洛传播、工况参数层经线性化叠加后, 分别计算其变异系数 CV_j^{pert} 与越界失效概率 $P_{fail,j}$, 结果对比如表 15 所示:

表 15 设计方案在 $\pm 5\%$ 扰动下的稳定性对比

方案	CV_1^{pert}	CV_2^{pert}	CV_3^{pert}	P_{fail}
综合最优方案 $\mathbf{X}^*$	0.27%	1.76%	0.31%	0.00%
鲁棒方案 $\mathbf{X}_{robust}$	0.26%	1.88%	0.32%	0.00%

若鲁棒方案的变异系数与失效概率均不劣于综合最优方案, 且满足 $CV_j^{pert} < 3\%$ 、

$P_{fail,j} \approx 0$ 的判定准则, 则表明鲁棒方案在加工误差与工况波动下依然保持稳定。

六、模型的评价及推广

6.1 模型检验

6.1.1 代理模型精度检验

问题二建立的 GPR 代理模型采用留出法评估拟合与预测精度, 训练集与测试集按 8 : 2 划分, 独立测试样本不参与训练。三项性能指标的检验结果见表 16。由表可见, 三个模型的决定系数 R^2 均接近于 1, 均方根误差与平均绝对百分比误差均处于极低量级, 表明代理模型对仿真数据的拟合与预测精度高, 可代替 CFD 求解器用于后续的多目标优化与不确定性传播分析。

表 16 GPR 代理模型精度检验结果 (测试集)

性能指标	R^2	$RMSE$	$MAPE$
热阻 R_{th}	1.0000	9.95×10^{-6}	9.83×10^{-6}
压降 ΔP	0.9997	4.77×10^{-4}	3.60×10^{-3}
温度非均匀性 ΔT	0.9998	1.63×10^{-4}	1.56×10^{-4}

6.1.2 机理规律与仿真数据的一致性检验

问题一推导的参数影响规律与附件 2 样本数据的控制变量分析结果对照: 针肋宽度比对热阻的影响在样本范围内呈“先降后升”的非单调变化, 各组控制变量下极小值均出现在 $\alpha \approx 0.2$ 附近; 歧管深高比增大使流量分配更均匀, 温度非均匀性总体下降且影响幅度较小; 针肋排数增大使压降总体升高, 且增幅随宽度比增大而加大, 均与机理推导一致, 表明机理模型对系统行为的刻画符合仿真规律。定量吻合程度方面, 压降关联式在全样本上的平均相对偏差为 6.02% ($R^2 = 0.8769$), 热阻关联式的平均相对偏差为 0.59% ($R^2 = 0.7486$); 热阻 R^2 偏低的主因是无量纲热阻量程窄 (0.72–0.77), R^2 对该量程天然不敏感, 而 MAPE 仅 0.59% 表明关联式的绝对拟合精度良好。关联式定位为机理方向验证与 α^* 论证, 定量预测职能由 GPR 代理模型 (6.1.1 节) 承担, 分工合理。

6.1.3 优化结果的可行性检验

将问题三所得综合最优方案回代 GPR 代理模型验证目标函数值, 与优化过程中预测值的相对偏差小于 0.05% (热阻 0.0009%、压降 0.0445%、温度非均匀性 0.0000%), 验证了优化结果的可行性与自洽性。

6.2 灵敏度分析

结构参数与工况参数对性能指标的敏感性已在问题五中系统完成: 结构参数扰动对三项指标的影响由训练好的代理模型严格刻画, 采用 Sobol 全局敏感性指数 (含一阶主效应与总效应) 与局部归一化敏感性系数定量评估, 并基于蒙特卡洛不确定性传播给出变异系数、95% 置信上限与越界失效概率; 工况参数 (流量、入口温度、发热功率) 的影响基于无量纲标度关系的机理线性化近似评估, 详见 5.5 节。分析表明, 温度非均匀性对针肋排数 n 的扰动最为敏感 (Sobol 总效应 $S_T = 0.7210$); 三个结构参数对至少一项指标的总效应指数均超过 0.2 阈值, 均为系统关键控制参数, 加工公差均须严格控制 (建议光刻线宽误差 $\leq \pm 1\%$, 蚀刻深度误差 $\leq \pm 3\%$); 在 $\pm 5\%$ 综合扰动下, 三项性能指标的变异系数均低于 3% 且越界失效概率均为零, 设计方案具有充分的工程鲁棒性。该结论为加工公差控制与运行工况约束提供了定量依据。

6.3 模型的优缺点

本文采用机理建模与数据驱动融合的技术路线, 整体框架完整、工程指标可直接落地, 同时亦存在若干局限, 分述如下。

6.3.1 模型的优点

(1) 机理与数据融合。问题一建立 CFD 机理模型, 问题二以 GPR 代理模型逼近高维非线性映射, 兼顾物理可解释性与拟合精度。

(2) 概率框架天然契合。GPR 除点预测外还提供预测方差, 为后续优化与不确定性传播提供天然的概率框架, 无需额外建模。

(3) 优化决策链条完整。NSGA-II 全局搜索与 L-BFGS-B 局部精化结合、针肋排数整数离散修正, 保证全局性与收敛精度; 理想点法消除量纲差异给出唯一综合方案, Minimax 后悔值准则降低方案对偏好权重的依赖。

(4) 敏感性评估体系完备。局部与全局敏感性分析 (Sobol) 结合, 给出变异系数、越界失效概率等工程可操作指标, 直接服务于制造公差与运行工况约束设计。

6.3.2 模型的缺点

(1) GPR 外推能力受限。预测精度依赖训练样本的网格覆盖范围, 外推至样本空间之外或极端工况时精度难以保证。

(2) 机理模型存在简化假设。均匀体热源、忽略热辐射等假设与实际芯片封装结构的传热过程存在偏差。

(3) 不确定性传播计算代价大。蒙特卡洛传播的收敛性依赖采样规模, 参数维度升高时计算代价随之增大。

(4) 工况敏感性评估范围有限。基于无量纲标度关系的线性外推仅在 $\pm 5\%$ 小扰动范围内有效, 扰动幅度增大时误差随之增大。

(5) **Minimax 准则**偏保守。在极端偏好场景下, 鲁棒方案的综合性能可能略逊于针对性优化方案。

6.4 模型推广

本文建立的“机理建模——代理模型——多目标优化——鲁棒决策——敏感性评估”完整框架具有良好通用性。在散热领域, 该框架可直接推广至液冷板、均温板、浸没式冷却等其他电子散热构型的参数化设计与优化; 对微通道结构参数与性能指标的映射关系, 同样适用于动力电池热管理、数据中心冷却系统的多目标优化。在方法论层面, GPR 代理模型驱动的昂贵仿真优化范式适用于任何依赖 CFD、有限元等数值仿真的设计问题; 基于 Sobol 全局敏感性分析与蒙特卡洛传播的稳健性评估框架, 亦可推广至制造公差设计、工艺参数稳健优化等工程场景, 为“结构—性能—稳健性”一体化设计提供通用解决方案。

参考文献

- [1] Tuckerman D B, Pease R F W. High-performance heat sinking for VLSI[J]. IEEE Electron Device Letters, 1981, 2(5): 126–129.
- [2] White F M. Fluid Mechanics (8th ed.)[M]. New York: McGraw-Hill, 2016.
- [3] 杨世铭, 陶文铨. 传热学 (第 4 版)[M]. 北京: 高等教育出版社, 2006.
- [4] Rasmussen C E, Williams C K I. Gaussian Processes for Machine Learning[M]. Cambridge: MIT Press, 2006.
- [5] Hwang C L, Yoon K. Multiple Attribute Decision Making: Methods and Applications[M]. Berlin: Springer-Verlag, 1981.
- [6] Deb K, Pratap A, Agarwal S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182–197.
- [7] Byrd R H, Lu P, Nocedal J, et al. A limited memory algorithm for bound constrained optimization[J]. SIAM Journal on Scientific Computing, 1995, 16(5): 1190–1208.
- [8] 陈宝林. 最优化理论与算法 (第 2 版)[M]. 北京: 清华大学出版社, 2005.
- [9] Savage L J. The theory of statistical decision[J]. Journal of the American Statistical Association, 1951, 46(253): 55–67.

- [10] Sobol I M. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates[J]. Mathematics and Computers in Simulation, 2001, 55(1–3): 271–280.
- [11] McKay M D, Beckman R J, Conover W J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code[J]. Technometrics, 1979, 21(2): 239–245.
- [12] 陶文铨. 数值传热学 (第 2 版)[M]. 西安: 西安交通大学出版社, 2001.

七、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模思路讨论、代码编写与调试、论文文本编辑与图表设计，详细使用情况见支撑材料。

附录 A 项目源代码

以下为问题一至问题五建模所用的全部 Python 源程序, 按模块依赖关系排列: 公共模块 → 逐题求解 → 画图脚本。

1.1 common.py —公共配置模块

项目级路径、matplotlib 全局样式(中文字体、DPI、网格)及统一图片保存函数。所有求解和画图脚本均依赖此模块。

Listing 1: common.py

```
1 # =====
2 # common.py — 公共配置:路径、matplotlib 样式、通用工具
3 #
4 # 所有脚本都以 code/ 为当前工作目录运行(make fig 保证),
5 # 统一从这里取路径,不要自己拼相对路径。
6 # =====
7 from pathlib import Path
8
9 # 路径
10 CODE_DIR = Path(__file__).resolve().parent.parent # code/
11 DATA_DIR = CODE_DIR / "data" # code/data/
12 FIG_DIR = CODE_DIR / "figures" # code/figures/ (输出目录)
13
14 FIG_DIR.mkdir(exist_ok=True)
15
16 # ----- matplotlib 全局样式(统一图风格,避免每人一版) -----
17 import matplotlib
18
19 matplotlib.use("Agg") # 服务器/无界面环境也能出图
20
21 import matplotlib.pyplot as plt
22 from matplotlib import font_manager
23
24 # 中文字体优先级:Noto Sans CJK SC > Noto Sans SC > Source Han Sans SC > SimHei > Microsoft YaHei
25 # 取第一个系统已安装的字体,找不到才用默认(此时中文会显示方框)
26 _FONT_CANDIDATES = [
27     "Noto Sans CJK SC", "Noto Sans SC", "Source Han Sans SC",
28     "Source Han Sans CN", "SimHei", "Microsoft YaHei",
29 ]
30 _installed = {f.name for f in font_manager.fontManager.ttflist}
31 _zh_font = next((f for f in _FONT_CANDIDATES if f in _installed), None)
32
33 plt.rcParams.update({
34     "font.family": _zh_font if _zh_font else "sans-serif", # 中文字体
35     "axes.unicode_minus": False, # 解决负号显示为方块的问题
36     "figure.dpi": 150, # 出图清晰度(国赛要求 300dpi 打印,提交前调高重跑)
37     "savefig.dpi": 300, # 论文印刷 300 dpi
38     "figure.figsize": (8, 5),
39     "axes.grid": True,
40     "grid.alpha": 0.3,
41     "grid.linewidth": 0.6,
```

```

42     "axes.linewidth": 0.8,      # 坐标轴线略细,出版级观感
43     "axes.titlesize": 12,
44     "axes.labelsizes": 11,
45     "xtick.labelsizes": 10,
46     "ytick.labelsizes": 10,
47     "legend.fontsize": 9,
48     "legend.framealpha": 0.9,
49     "lines.linewidth": 1.6,
50 })
51
52 # Okabe-Ito 色盲安全配色(统一全项目用色,避免每人一版颜色)
53 # 语义约定:
54 # 指标:Rth=蓝 dP=橙 dT=绿
55 # 结构参数:x1( $\alpha$ )=蓝 x2( $\beta$ )=橙 x3( $n$ )=绿
56 # 方案对比:X*=蓝 X_robust=橙
57 OKABE_ITO = ["#0072B2", "#D55E00", "#009E73", "#CC79A7",
58             "#E69F00", "#56B4E9", "#F0E442", "#000000"]
59 COLOR_RTH, COLOR_DP, COLOR_DT = OKABE_ITO[0], OKABE_ITO[1], OKABE_ITO[2]
60
61
62 def savefig(name: str) -> Path:
63     """保存图片到 code/figures/,make fig 会自动同步到 paper/figures/。
64
65     命名规范:fig_<内容>.png,例如 fig_q1.png、fig_sensitivity.png
66     """
67     path = FIG_DIR / name
68     plt.savefig(path, bbox_inches="tight")
69     plt.close()
70     print(f"[fig] saved -> {path}")
71     return path

```

1.2 solve_q1.py —问题一: 半经验关联式标定与机理分析

- (1) 压降幂律关联式 $\ln \Delta P^* = \ln C + a \ln \beta + b \ln(1 - \alpha) + c \ln n$ 的最小二乘标定
- (2) 热阻二次结构关联式 $R_{th}^* = R_0 + c_1(\alpha - \alpha^*)^2 + c_2\beta + c_3n + c_4\alpha n$ 的非线性拟合
- (3) 基于 GPR 代理模型的控制变量影响幅度定量表
- (4) α^* 拐点的 GPR 交叉验证 (逐点扫描求 $\bar{R}_{th}^*$ 的 argmin)
- (5) 机理关联式与数据一致性定量检验 (MAPE, R^2)

Listing 2: solve_q1.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4 import common
5
6 import numpy as np
7 import pandas as pd
8 import joblib as jb
9 from scipy.optimize import curve_fit
10 from sklearn.metrics import r2_score, mean_absolute_percentage_error
11
12 file = common.DATA_DIR / "data.xlsx"
13 data = pd.read_excel(file, header=1).rename(columns={
14     "样本编号": "index",
15     "针肋宽度比": "alpha",
16     "歧管深高比": "beta",
17     "单个歧管单元内沿流向的针肋排数": "n",
18     "无量纲热阻": "Rth",
19     "无量纲压降": "dP",
20     "无量纲温度非均匀性": "dT",
21 })
22
23 # ----- 1.1 半经验关联式标定 -----
24 # dP* = C * beta^a * (1-alpha)^b * n^c : 幂律, 取对数后线性最小二乘。
25 # n=0(无针肋)的4个样本不满足幂律结构(没有"逐排叠加"的物理机制), 排除在此拟合外。
26 FINNED = data[data["n"] > 0].copy()
27
28
29 def fit_dp_corr(df):
30     y = np.log(df["dP"].to_numpy())
31     X = np.column_stack([
32         np.ones(len(df)),
33         np.log(df["beta"].to_numpy()),
34         np.log(1 - df["alpha"].to_numpy()),
35         np.log(df["n"].to_numpy()),
36     ])
37     coef, *_ = np.linalg.lstsq(X, y, rcond=None)
38     lnC, a, b, c = coef
39     C = np.exp(lnC)
40     dp_pred = C * df["beta"] ** a * (1 - df["alpha"]) ** b * df["n"] ** c
41     r2 = r2_score(df["dP"], dp_pred)
42     mape = mean_absolute_percentage_error(df["dP"], dp_pred)
43     return {"C": C, "a": a, "b": b, "c": c, "r2": r2, "mape": mape, "pred": dp_pred}
```



```

44
45
46 def rth_model(X, R0, c1, c2, c3, c4, alpha_star):
47     alpha, beta, n = X
48     # 基础式(eq:rth-corr)只有R0/c1/c2/c3四项时R2=0.61(<0.9),FILL_CHECKLIST建议
49     # 加alpha*n交互项(排数对热阻的边际影响随宽度比变化)——加了之后R2升到0.75,
50     # 仍不到0.9但明显改善,MAPE本来就很低(<1%,Rth数值范围本身很窄0.72~0.77,
51     # R2对窄量程数据天然不友好);已按建议加了交互项,R2仍偏低的情况在notes.md
52     # 里记录,留给建模手判断是否需要进一步改结构。
53     return R0 + c1 * (alpha - alpha_star) ** 2 + c2 * beta + c3 * n + c4 * alpha * n
54
55
56 def fit_rth_corr(df):
57     X = (df["alpha"].to_numpy(), df["beta"].to_numpy(), df["n"].to_numpy())
58     y = df["Rth"].to_numpy()
59     p0 = [0.75, 0.1, 0.0, 0.0, 0.0, 0.2] # alpha*初值取拐点附近0.2(论文预期)
60     popt, _ = curve_fit(rth_model, X, y, p0=p0, maxfev=20000)
61     R0, c1, c2, c3, c4, alpha_star = popt
62     rth_pred = rth_model(X, *popt)
63     r2 = r2_score(y, rth_pred)
64     mape = mean_absolute_percentage_error(y, rth_pred)
65     return {"R0": R0, "c1": c1, "c2": c2, "c3": c3, "c4": c4, "alpha_star": alpha_star,
66           "r2": r2, "mape": mape, "pred": rth_pred}
67
68
69 # ----- 1.2 影响幅度定量表(用GPR代理模型做控制变量分析,数据网格不满不能直接分组平均) -----
70
71 res = jb.load(common.DATA_DIR / "gpr.pkl")
72 scaler = res["_scaler"]
73 gpr_outputs = [k for k in res.keys() if not k.startswith("_")]
74 gprs = {col: res[col]["gpr"] for col in gpr_outputs}
75
76
77 def avg_relative_change(alpha1, alpha2, beta_grid, n_grid, out_col):
78     """固定(beta,n)遍历网格,统计 out_col 从 alpha1 到 alpha2 的平均相对变化(%)"""
79     changes = []
80     for beta in beta_grid:
81         for n in n_grid:
82             X = np.array([[alpha1, beta, n], [alpha2, beta, n]])
83             X_std = scaler.transform(X)
84             y = gprs[out_col].predict(X_std)
85             changes.append((y[1] - y[0]) / y[0] * 100)
86     return np.mean(changes)
87
88
89 def influence_table():
90     BETA_G = [3.0, 3.5, 4.0, 4.5]
91     N_G = [2, 4, 6, 8, 10]
92     ALPHA_G = [0.10, 0.15, 0.20, 0.30]
93
94     rows = {}
95     # alpha 影响: Rth先降后升, dP总体增大
96     rows["alpha_010_020_Rth"] = avg_relative_change(0.10, 0.20, BETA_G, N_G, "Rth")
97     rows["alpha_020_030_Rth"] = avg_relative_change(0.20, 0.30, BETA_G, N_G, "Rth")
98     rows["alpha_010_030_dP"] = avg_relative_change(0.10, 0.30, BETA_G, N_G, "dP")
99     # beta 影响: dP、dT总体减小

```

```

100 rows["beta_300_450_dP"] = avg_relative_change_param(3.00, 4.50, "beta", ALPHA_G, N_G, "dP")
101 rows["beta_300_450_dT"] = avg_relative_change_param(3.00, 4.50, "beta", ALPHA_G, N_G, "dT")
102 # n 影响: Rth下降, dP增大(按alpha分档)
103 rows["n_2_10_Rth"] = avg_relative_change_param(2, 10, "n", ALPHA_G, BETA_G, "Rth")
104 rows["n_2_10_dP_by_alpha"] = {
105     a: avg_relative_change_param(2, 10, "n", [a], BETA_G, "dP")
106     for a in ALPHA_G
107 }
108 return rows
109
110
111 def avg_relative_change_param(v1, v2, which, grid_a, grid_b, out_col):
112     """通用版:把 which('beta'或'n') 从v1变到v2,固定另外两个参数遍历grid_a x grid_b
113     (which='n'时,grid_a是alpha取值,grid_b是beta取值)"""
114     changes = []
115     for a in grid_a:
116         for b in grid_b:
117             if which == "beta":
118                 alpha1, beta1, n1 = a, v1, b
119                 alpha2, beta2, n2 = a, v2, b
120             else: # which == "n"
121                 alpha1, beta1, n1 = a, b, v1
122                 alpha2, beta2, n2 = a, b, v2
123             X = np.array([[alpha1, beta1, n1], [alpha2, beta2, n2]])
124             X_std = scaler.transform(X)
125             y = gprs[out_col].predict(X_std)
126             changes.append((y[1] - y[0]) / y[0] * 100)
127     return np.mean(changes)
128
129
130 # ----- 1.3 alpha*拐点交叉验证(GPR直接求argmin,比半经验式更可信) -----
131
132 def gpr_alpha_star(beta_grid=(3.0, 3.5, 4.0, 4.5), n_grid=(2, 4, 6, 8, 10), n_points=41):
133     """在GPR(R2>0.999)上直接对alpha扫描求Rth平均值的argmin,作为alpha*的交叉验证"""
134     alphas = np.linspace(0.10, 0.30, n_points)
135     avg = []
136     for a in alphas:
137         vals = [gprs["Rth"].predict(scaler.transform([[a, b, n]]))[0]
138                 for b in beta_grid for n in n_grid]
139         avg.append(np.mean(vals))
140     avg = np.array(avg)
141     i = np.argmin(avg)
142     return alphas[i], avg[i]
143
144
145 # ----- 1.4 机理关联式与数据一致性定量检验 -----
146
147 def mechanism_data_agreement(dp_fit, rth_fit):
148     """机理关联式(1.1标定)在样本上的平均相对偏差,作为6.1.2定量吻合指标"""
149     return {
150         "dp_mape_pct": dp_fit["mape"] * 100,
151         "rth_mape_pct": rth_fit["mape"] * 100,
152         "dp_r2": dp_fit["r2"],
153         "rth_r2": rth_fit["r2"],
154     }
155

```

```

156
157 def solve():
158     dp_fit = fit_dp_corr(FINNED)
159     rth_fit = fit_rth_corr(data)
160
161     print("== 1.1 半经验关联式标定 ==")
162     print(f"dp* = {dp_fit['C']:.4f} * beta^{dp_fit['a']:.4f} * (1-alpha)^{dp_fit['b']:.4f} * n^{
        ↪ dp_fit['c']:.4f}")
163     print(f" R2={dp_fit['r2']:.4f} MAPE={dp_fit['mape']*100:.2f}% (n>0的{len(FINNED)}个样本)")
164     print(f"Rth* = {rth_fit['R0']:.4f} + {rth_fit['c1']:.4f}*(alpha-{rth_fit['alpha_star']:.4f})^2 "
        f"+ {rth_fit['c2']:.4f}*beta + {rth_fit['c3']:.4f}*n + {rth_fit['c4']:.4f}*alpha*n")
165     print(f" R2={rth_fit['r2']:.4f} MAPE={rth_fit['mape']*100:.2f}% (全部{len(data)}个样本)")
166
167
168     print("\n== 1.2 影响幅度定量表(基于GPR代理模型,控制变量网格平均) ==")
169     inf = influence_table()
170     print(f"alpha 0.10->0.20: Rth平均变化 {inf['alpha_010_020_Rth']:.2f}%")
171     print(f"alpha 0.20->0.30: Rth平均变化 {inf['alpha_020_030_Rth']:.2f}%")
172     print(f"alpha 0.10->0.30: dP平均变化 {inf['alpha_010_030_dP']:.2f}%")
173     print(f"beta 3.00->4.50: dP平均变化 {inf['beta_300_450_dP']:.2f}%")
174     print(f"beta 3.00->4.50: dT平均变化 {inf['beta_300_450_dT']:.2f}%")
175     print(f"n 2->10: Rth平均变化 {inf['n_2_10_Rth']:.2f}%")
176     print(f"n 2->10: dP平均变化(按alpha分档):")
177     for a, v in inf["n_2_10_dP_by_alpha"].items():
178         print(f" alpha={a}: {v:.2f}%")
179
180     print("\n== 1.4 机理关联式与数据一致性(6.1.2定量吻合指标) ==")
181     agree = mechanism_data_agreement(dp_fit, rth_fit)
182     print(f"dp关联式: R2={agree['dp_r2']:.4f} 平均相对偏差={agree['dp_mape_pct']:.2f}%")
183     print(f"Rth关联式: R2={agree['rth_r2']:.4f} 平均相对偏差={agree['rth_mape_pct']:.2f}%")
184     print(f"alpha*(半经验式标定值) = {rth_fit['alpha_star']:.4f}")
185     alpha_star_gpr, rth_at_star = gpr_alpha_star()
186     print(f"alpha*(GPR交叉验证,argmin,更可信) = {alpha_star_gpr:.4f} (对应Rth均值={rth_at_star:.4f});"
        f"论文预期在0.2附近,以此值为准)")
187
188
189     out = {
190         "dp_fit": dp_fit, "rth_fit": rth_fit,
191         "influence": inf, "agreement": agree,
192         "alpha_star_gpr": alpha_star_gpr,
193     }
194     jb.dump(out, common.DATA_DIR / "q1_result.pkl")
195     return out
196
197
198 if __name__ == "__main__":
199     solve()

```

1.3 solve_q2.py —问题二:GPR 代理模型构建

以 α, β, n 为输入、 $R_{th}^*, \Delta P^*, \Delta T^*$ 为输出, 采用 Constant $\times$ RBF(各向异性)+White 复合核构建高斯过程回归代理模型。按 8:2 留出法划分训练/测试集, 对每个输出维度独立训练 (MISO 架构), 并提取 MLE 最优核超参数 $\theta^* = [\sigma_f, l_1, l_2, l_3, \sigma_n]$ 。训练完成的模型保存为 `gpr.pkl`, 供后续问题三–五复用。

Listing 3: solve_q2.py

```
1 import sys
2 from pathlib import Path
3
4 sys.path.insert(0, str(Path(__file__).resolve().parent))
5 import common
6 import numpy as np
7 import pandas as pd
8 import joblib as jb
9 from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import StandardScaler
11 from sklearn.gaussian_process.kernels import WhiteKernel, RBF, ConstantKernel
12 from sklearn.gaussian_process import GaussianProcessRegressor
13 from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_percentage_error
14
15
16 file = common.DATA_DIR / "data.xlsx"
17 data = pd.read_excel(file, header=1).rename(columns={
18     "样本编号": "index",
19     "针肋宽度比": "alpha",
20     "歧管深高比": "beta",
21     "单个歧管单元内沿流向的针肋排数": "n",
22     "无量纲热阻": "Rth",
23     "无量纲压降": "dP",
24     "无量纲温度非均匀性": "dT",
25 })
26 inputs = ["alpha", "beta", "n"]
27 outputs = ["Rth", "dP", "dT"]
28
29
30 def gpr():
31     #每次选中一个输出维度进行训练,先进行总的一次切分,后续单独训练
32     X_train, X_test, y_train, y_test = train_test_split(data[inputs], data[outputs], test_size=0.2,
33         ↳ random_state=676767)
34
35     # Z-score-scaler
36     scaler = StandardScaler()
37     X_train_std = scaler.fit_transform(X_train)
38     X_test_std = scaler.transform(X_test)
39
40     #kernel
41     kernel = ConstantKernel(1.0, (1e-10, 1e10)) * RBF([1.0, 1.0, 1.0], (1e-10, 1e10)) + WhiteKernel
42         ↳ (1.0, (1e-10, 1e10))
43
44     #GPR
45     result = {}
```

```

44 for y_train_select_col in outputs:
45     y_train_select = y_train[y_train_select_col]
46     gpr = GaussianProcessRegressor(kernel=kernel, alpha=1e-8, normalize_y=True,
47                                     ⇨ n_restarts_optimizer=10)
48     gpr.fit(X_train_std, y_train_select)
49     mu, sigma = gpr.predict(X_test_std, return_std=True)
50
51     # 提取MLE寻优后的核超参数 theta*=[sigma_f, l1,l2,l3, sigma_n](eq:kernel,04-models.tex:264待回填)
52     k_const_rbf, k_white = gpr.kernel_.k1, gpr.kernel_.k2
53     sigma_f = np.sqrt(k_const_rbf.k1.constant_value)
54     length_scales = k_const_rbf.k2.length_scale # [l1, l2, l3] 对应 alpha, beta, n
55     sigma_n = np.sqrt(k_white.noise_level)
56
57     result[y_train_select_col] = {
58         "gpr": gpr,
59         "y_true": y_test[y_train_select_col].to_numpy(),
60         "y_pred": mu,
61         "y_std": sigma,
62         "r2": r2_score(y_test[y_train_select_col], mu),
63         "rmse": mean_squared_error(y_test[y_train_select_col], mu) ** 0.5,
64         "mape": mean_absolute_percentage_error(y_test[y_train_select_col], mu),
65         "theta_star": {"sigma_f": sigma_f, "length_scales": length_scales, "sigma_n": sigma_n},
66     }
67     #保存GPR模型文件(连同scaler一起,Q3要用同一套标准化)
68 result["_scaler"] = scaler
69 result["_input_cols"] = inputs
70 jb.dump(result, common.DATA_DIR / "gpr.pkl")
71 print("duplicated data: ", data.duplicated(subset=inputs).sum(), "\n")
72 for col in outputs:
73     r = result[col]
74     print(f"{col}: r2={r['r2']:.8f} rmse={r['rmse']:.8f} mape={r['mape']:.8f}")
75 print("\n== GPR最优超参数 theta*=[sigma_f, l_alpha, l_beta, l_n, sigma_n] ==")
76 for col in outputs:
77     t = result[col]["theta_star"]
78     l1, l2, l3 = t["length_scales"]
79     print(f"{col}: sigma_f={t['sigma_f']:.4f} l_alpha={l1:.4f} l_beta={l2:.4f} l_n={l3:.4f} "
80           f"sigma_n={t['sigma_n']:.6f}")
81 return result
82
83 if __name__ == "__main__":
84     gpr()

```

1.4 solve_q3.py —问题三: 混合多目标优化与决策

三段式混合策略:

- (1) 混合变量 NSGA-II (mode 变量处理 $\alpha = 0 \Leftrightarrow n = 0$ 的耦合约束), 多种子拼接 Pareto 前沿
- (2) 在全体合法 (β, n) 离散组合上穷举, L-BFGS-B 精修 α
- (3) 均等权重理想点法选出综合最优方案 $\mathbf{X}^*$

Listing 4: solve_q3.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4 import common
5
6 import numpy as np
7 import joblib as jb
8 from scipy.optimize import minimize as lbfgsb
9
10 from pymoo.core.problem import Problem
11 from pymoo.core.variable import Real, Integer, Choice
12 from pymoo.core.mixed import (
13     MixedVariableSampling,
14     MixedVariableMating,
15     MixedVariableDuplicateElimination,
16 )
17 from pymoo.algorithms.moo.nsga2 import NSGA2
18 from pymoo.optimize import minimize as pymoo_minimize
19
20 res = jb.load(common.DATA_DIR / "gpr.pkl")
21 scaler = res["_scaler"]
22 outputs = [k for k in res.keys() if not k.startswith("_")] # ["Rth", "dP", "dT"], 顺序与 W 对应
23 gprs = {col: res[col]["gpr"] for col in outputs}
24
25 BETA_LEVELS = [3.0, 3.5, 4.0, 4.5] # 附件2里beta实际只有这4档离散取值
26
27 # 可行域(04-models.tex 5.3.1,eq:q3-constraints,main分支更新版):
28 # x1 in {0} U [0.10, 0.30], x3 in {0} U {2,...,10}, 耦合: alpha=0 <=> n=0
29 X1_FINNED_BOUNDS = (0.10, 0.30)
30 X3_FINNED_RANGE = range(2, 11)
31 W = np.array([1 / 3, 1 / 3, 1 / 3]) # 权重,对应 (Rth, dP, dT)
32
33 # 全体合法离散(x2,x3)组合: 无针肋模式(x3=0,搭配x1=0) + 有针肋模式(x3=2..10,x1连续可调)
34 ALL_COMBOS = (
35     [(x2, 0) for x2 in BETA_LEVELS]
36     + [(x2, x3) for x2 in BETA_LEVELS for x3 in X3_FINNED_RANGE]
37 )
38
39
40 def predict(X: np.ndarray) -> np.ndarray:
41     """X: (n, 3) 原始尺度 [alpha, beta, n] -> F: (n, 3) [Rth, dP, dT]"""
42     X_std = scaler.transform(X)
43     return np.column_stack([gprs[col].predict(X_std) for col in outputs])
```

```

44
45
46 # ----- 第一步:全局多目标搜索(混合变量 NSGA-II,mode 变量处理耦合约束) -----
47
48 def to_physical(x):
49     """x: pymoo mixed-variable dict {mode, x1, x2, x3} -> [alpha, beta, n] 物理坐标"""
50     if x["mode"] == 0:
51         return [0.0, x["x2"], 0]
52     return [x["x1"], x["x2"], x["x3"]]
53
54
55 class SinkOptProblem(Problem):
56     def __init__(self):
57         super().__init__(
58             vars={
59                 "mode": Choice(options=[0, 1]), # 0=无针肋, 1=有针肋
60                 "x1": Real(bounds=X1_FINNED_BOUNDS),
61                 "x2": Choice(options=BETA_LEVELS),
62                 "x3": Integer(bounds=(X3_FINNED_RANGE.start, X3_FINNED_RANGE.stop - 1)),
63             },
64             n_obj=3,
65         )
66
67     def _evaluate(self, X, out, *args, **kwargs):
68         arr = np.array([to_physical(x) for x in X])
69         out["F"] = predict(arr)
70
71
72 def run_nsga2(pop_size=100, n_gen=200, seed=676767):
73     problem = SinkOptProblem()
74     algorithm = NSGA2(
75         pop_size=pop_size,
76         sampling=MixedVariableSampling(),
77         mating=MixedVariableMating(eliminate_duplicates=MixedVariableDuplicateElimination()),
78         eliminate_duplicates=MixedVariableDuplicateElimination(),
79     )
80     result = pymoo_minimize(
81         problem, algorithm,
82         termination=("n_gen", n_gen),
83         seed=seed,
84         verbose=False,
85     )
86     X_front = np.array([to_physical(x) for x in result.X])
87     F_front = result.F
88     return X_front, F_front
89
90
91 def run_nsga2_multiseed(pop_size=200, n_gen=300, seeds=range(10)):
92     """单次 NSGA-II 前沿不稳定(存在多个局部盆地),多种子跑完拼接前沿。"""
93     X_all, F_all = [], []
94     for seed in seeds:
95         X_front, F_front = run_nsga2(pop_size=pop_size, n_gen=n_gen, seed=seed)
96         X_all.append(X_front)
97         F_all.append(F_front)
98     return np.vstack(X_all), np.vstack(F_all)
99

```

```

100
101 # ----- 第二步:归一化基准(GPR 在合法可行域内的预测值域) -----
102
103 def gpr_value_range(n1=25):
104     """在合法可行域(无针肋点集 + 有针肋规则网格)上取 GPR 预测值的 min/max"""
105     x2_arr = np.array(BETA_LEVELS)
106     X_finless = np.column_stack([np.zeros_like(x2_arr), x2_arr, np.zeros_like(x2_arr)])
107
108     x1 = np.linspace(*X1_FINNED_BOUNDS, n1)
109     x3 = np.arange(X3_FINNED_RANGE.start, X3_FINNED_RANGE.stop)
110     g1, g2, g3 = np.meshgrid(x1, x2_arr, x3, indexing="ij")
111     X_finned = np.column_stack([g1.ravel(), g2.ravel(), g3.ravel()])
112
113     F = predict(np.vstack([X_finless, X_finned]))
114     return F.min(axis=0), F.max(axis=0)
115
116
117 def normalize(F, f_min, f_max):
118     return (F - f_min) / (f_max - f_min)
119
120
121 # ----- 第三步:理想点法/加权综合最优 —— 穷举全部合法(x2,x3)组合 + L-BFGS-B精修x1 -----
122
123 def ideal_point_distance(F_norm, w=W):
124     return np.sqrt(np.sum(w * F_norm ** 2, axis=1))
125
126
127 def refine_at_fixed_x2x3(x2, x3, x1_init, f_min, f_max, w=W):
128     """固定 (x2, x3): x3=0 时 x1 必须=0(无针肋,直接求值);否则对 x1 在 [0.10,0.30] 做 L-BFGS-B 精修"""
129     def D_of(x1_val):
130         F = predict(np.array([[x1_val, x2, x3]]))
131         F_norm = (F - f_min) / (f_max - f_min)
132         return np.sqrt(np.sum(w * F_norm ** 2))
133
134     if x3 == 0:
135         return 0.0, D_of(0.0)
136
137     def obj(x1):
138         return D_of(x1[0])
139
140     x0 = min(max(x1_init, X1_FINNED_BOUNDS[0]), X1_FINNED_BOUNDS[1])
141     result = lbfgsb(obj, x0=[x0], bounds=[X1_FINNED_BOUNDS], method="L-BFGS-B")
142     return result.x[0], result.fun
143
144
145 def global_best(w, f_min, f_max, x1_inits=(0.10, 0.15, 0.20, 0.25, 0.30)):
146     """穷举全部40种合法(x2,x3)组合,每种对x1多起点精修,取全局D最小 -> (x1*,x2*,x3*,D*)"""
147     best = None
148     for x2, x3 in ALL_COMBOS:
149         inits = (0.0,) if x3 == 0 else x1_inits
150         for x1_init in inits:
151             x1_r, D_r = refine_at_fixed_x2x3(x2, x3, x1_init, f_min, f_max, w=w)
152             if best is None or D_r < best[3]:
153                 best = (x1_r, x2, x3, D_r)
154     return best
155

```



```

156
157 def solve(pop_size=200, n_gen=300, seeds=range(10)):
158     X_front, F_front = run_nsga2_multiseed(pop_size=pop_size, n_gen=n_gen, seeds=seeds)
159     f_min, f_max = gpr_value_range()
160
161     # NSGA-II前沿上按理想点距离选出的"预精修"最优,仅用于回代验证对照
162     F_norm = normalize(F_front, f_min, f_max)
163     D_front = ideal_point_distance(F_norm)
164     pre_idx = np.argmin(D_front)
165     x_pre = X_front[pre_idx]
166     D_pre = D_front[pre_idx]
167     F_pre = F_front[pre_idx]
168
169     # 穷举全部合法离散组合 + L-BFGS-B精修,得到最终综合最优方案
170     x1_star, x2_star, x3_star, D_star = global_best(W, f_min, f_max)
171     F_star = predict(np.array([[x1_star, x2_star, x3_star]]))[0]
172
173     # 回代验证偏差: GA前沿上的预精修解 vs 精修后的最终解,相对偏差(%)
174     rel_dev = np.abs(F_star - F_pre) / np.abs(F_pre) * 100
175
176     print(f"Pareto front size: {len(X_front)}")
177     print(f"pre-refine best on front: x1={x_pre[0]:.4f} x2={x_pre[1]:.2f} x3={int(x_pre[2])} D={D_pre  

178         ↪ :.6f}")
179     print(f"refined optimum X*: x1={x1_star:.4f} x2={x2_star:.2f} x3={x3_star} D={D_star:.6f}")
180     print(f"predicted Rth={F_star[0]:.4f} dP={F_star[1]:.4f} dT={F_star[2]:.4f}")
181     print(f"回代验证相对偏差(%): Rth={rel_dev[0]:.4f} dP={rel_dev[1]:.4f} dT={rel_dev[2]:.4f}")
182
183     out = {
184         "X_front": X_front, "F_front": F_front,
185         "f_min": f_min, "f_max": f_max,
186         "x_star": (x1_star, x2_star, x3_star),
187         "F_star": F_star,
188         "D_star": D_star,
189         "rel_dev_pct": rel_dev,
190     }
191     jb.dump(out, common.DATA_DIR / "q3_result.pkl")
192     return out
193
194 if __name__ == "__main__":
195     solve()

```

1.5 solve_q4.py —问题四: 鲁棒优化

- (1) 三类典型应用场景 (高性能计算、便携低功耗、高可靠工业) 与权重单纯形网格的扫描
- (2) 敏感性矩阵 $S_{ij} = \partial x_i / \partial w_j$ (中心差分)
- (3) 基于 Minimax Regret 准则的鲁棒方案搜索
- (4) 跨场景稳定性评估 (权重空间变异系数 CV)

复用 solve_q3 的 global_best 函数作为加权最优求解器。

Listing 5: solve_q4.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4 import common
5
6 import numpy as np
7 import joblib as jb
8
9 import solve_q3 as q3
10
11 # ----- 权重场景 -----
12
13 SCENARIOS = {
14     "高性能计算": np.array([0.6, 0.2, 0.2]),
15     "便携/低功耗设备": np.array([0.2, 0.6, 0.2]),
16     "高可靠工业器件": np.array([0.2, 0.2, 0.6]),
17 }
18
19
20 def simplex_grid(n=6):
21     """重心坐标网格: w = (i/n, j/n, (n-i-j)/n), i+j<=n, 天然满足 sum(w)=1"""
22     pts = []
23     for i in range(n + 1):
24         for j in range(n + 1 - i):
25             k = n - i - j
26             pts.append([i / n, j / n, k / n])
27     return np.array(pts)
28
29
30 # 给定权重求最优解:直接复用 solve_q3.global_best(合法(x2,x3)组合穷举 + 一维L-BFGS-B精修x1)
31 solve_weighted = q3.global_best
32
33
34 # ----- 第一部分(1)(2): 权重扫描 -----
35
36 def weight_sweep(weights, f_min, f_max):
37     """对一批权重逐个求最优解,返回 X*(W), D*(W)"""
38     X_star = np.zeros((len(weights), 3))
39     D_star = np.zeros(len(weights))
40     for i, w in enumerate(weights):
41         x1, x2, x3, D = solve_weighted(w, f_min, f_max)
```

```

42     X_star[i] = [x1, x2, x3]
43     D_star[i] = D
44     return X_star, D_star
45
46
47 # ----- 第一部分(3): 敏感性矩阵(单纯形内保约束的中心差分) -----
48
49 def perturb_weight(w0, j, h):
50     """沿第j个权重方向扰动 h,同时从另外两个权重里等量扣除,保持 sum(w)=1"""
51     dw = -h / 2 * np.ones(3)
52     dw[j] = h
53     w = w0 + dw
54     return w
55
56
57 def sensitivity_matrix(w0, f_min, f_max, h=0.05):
58     """S[i,j] = d x_i / d w_j, 中心差分, i in {x1,x2,x3}, j in {w1,w2,w3}"""
59     S = np.zeros((3, 3))
60     for j in range(3):
61         w_plus = perturb_weight(w0, j, h)
62         w_minus = perturb_weight(w0, j, -h)
63         if (w_plus < 0).any() or (w_minus < 0).any():
64             # 靠近单纯形边界时缩小步长,避免扰动出界
65             h_local = min(w0[j], (1 - w0[j]) / 2, h) * 0.5
66             w_plus = perturb_weight(w0, j, h_local)
67             w_minus = perturb_weight(w0, j, -h_local)
68             h_use = h_local
69         else:
70             h_use = h
71         x_plus = np.array(solve_weighted(w_plus, f_min, f_max)[:3])
72         x_minus = np.array(solve_weighted(w_minus, f_min, f_max)[:3])
73         S[:, j] = (x_plus - x_minus) / (2 * h_use)
74     return S
75
76
77 # ----- 第二部分: minimax regret 鲁棒设计 -----
78
79 def minimax_regret(candidates, weight_grid, f_min, f_max, D_star_grid):
80     """candidates: (n,3) 候选X池; weight_grid/D_star_grid: 权重采样点及其D*(W)
81     返回每个候选的最大后悔值,及取最大后悔值最小的鲁棒解"""
82     F_cand = q3.predict(candidates)
83     F_norm_cand = q3.normalize(F_cand, f_min, f_max) # (n_cand, 3)
84
85     max_regret = np.full(len(candidates), -np.inf)
86     for w, D_opt in zip(weight_grid, D_star_grid):
87         D_w = np.sqrt(np.sum(w * F_norm_cand ** 2, axis=1)) # (n_cand,)
88         regret = D_w - D_opt
89         max_regret = np.maximum(max_regret, regret)
90
91     robust_idx = np.argmin(max_regret)
92     return robust_idx, max_regret
93
94
95 def cv_of_candidate(X, weight_grid, f_min, f_max):
96     """CV(X) = std(D_W(X))/mean(D_W(X)), 在权重网格上评估单个候选X的稳健性(离散系数)"""
97     F = q3.predict(np.array([X]))[0]

```

```

98 F_norm = (F - f_min) / (f_max - f_min)
99 D_w = np.sqrt(np.sum(weight_grid * F_norm ** 2, axis=1))
100 return D_w.std() / D_w.mean()
101
102
103 def solve():
104     f_min, f_max = q3.gpr_value_range()
105
106     # ---- (1)(2) 权重扫描: 3个典型场景 + 单纯形网格 ----
107     scenario_names = list(SCENARIOS.keys())
108     scenario_w = np.array(list(SCENARIOS.values()))
109     grid_w = simplex_grid(n=6)
110     all_w = np.vstack([scenario_w, grid_w])
111
112     X_star, D_star = weight_sweep(all_w, f_min, f_max)
113
114     print("== 权重扫描: 3个典型场景 ==")
115     for name, w, x, d in zip(scenario_names, all_w[:3], X_star[:3], D_star[:3]):
116         print(f"{name} w={w}: x1={x[0]:.4f} x2={x[1]:.2f} x3={int(x[2])} D={d:.6f}")
117     print(f"单纯形网格采样点数: {len(grid_w)}")
118
119     # ---- (3) 敏感性矩阵: 在3个典型场景处各算一个 ----
120     print("\n== 敏感性矩阵 S[i,j]=dx_i/dw_j (在3个典型场景处) ==")
121     sens_matrices = {}
122     for name, w in zip(scenario_names, scenario_w):
123         S = sensitivity_matrix(w, f_min, f_max)
124         sens_matrices[name] = S
125         print(f"-- {name} --")
126         print(S)
127
128     # ---- 第二部分: minimax regret 鲁棒方案 ----
129     q3_res = jb.load(common.DATA_DIR / "q3_result.pkl")
130     candidates = q3_res["X_front"] # 复用Q3的Pareto前沿当候选池
131     D_star_equal = q3_res["D_star"] # Q3等权重(1/3,1/3,1/3)下的D*,作为epsilon基准
132
133     robust_idx, max_regret = minimax_regret(candidates, all_w, f_min, f_max, D_star)
134     x_robust = candidates[robust_idx]
135     R_max = max_regret[robust_idx]
136     epsilon = 0.05 * D_star_equal # eq:minimax: epsilon = 5% * D*(等权重Q3最优值)
137
138     cv_robust = cv_of_candidate(x_robust, all_w, f_min, f_max)
139     cv_equal = cv_of_candidate(np.array(q3_res["x_star"]), all_w, f_min, f_max)
140
141     print(f"\n== 鲁棒方案 (minimax regret) ==")
142     print(f"X_robust: x1={x_robust[0]:.4f} x2={x_robust[1]:.2f} x3={int(round(x_robust[2]))}")
143     print(f"R_max = {R_max:.6f} epsilon(5%*D*_Q3) = {epsilon:.6f} R_max<=epsilon: {R_max <= epsilon}"
144           ↪ )
145     print(f"CV(X_robust) = {cv_robust:.6f} CV(X*_Q3等权重解) = {cv_equal:.6f}")
146
147     out = {
148         "f_min": f_min, "f_max": f_max,
149         "scenario_names": scenario_names, "scenario_w": scenario_w,
150         "all_w": all_w, "X_star": X_star, "D_star": D_star,
151         "sens_matrices": sens_matrices,
152         "candidates": candidates, "max_regret": max_regret,
153         "x_robust": x_robust, "robust_idx": robust_idx,

```

```
153     "R_max": R_max, "epsilon": epsilon,  
154     "cv_robust": cv_robust, "cv_equal": cv_equal,  
155 }  
156 jb.dump(out, common.DATA_DIR / "q4_result.pkl")  
157 return out  
158  
159  
160 if __name__ == "__main__":  
161     solve()
```

1.6 solve_q5.py —问题五: 敏感性分析与不确定性传播

结构参数层 (GPR 直接支撑):

- Sobol 全局敏感性指数 (一阶 S_i 与总效应 S_{Ti} , $N = 1024$)
- 局部归一化敏感性系数 S_{ij}^{local} (中心差分)
- LHS 蒙特卡洛不确定性传播 ($N = 8000$, $\pm 5\%$ 扰动)

工况参数层 (机理标度近似): 基于无量纲传热/流动标度关系的一阶线性化估计。验证闭环: 对比综合最优方案 $\mathbf{X}^*$ 与鲁棒方案的变异系数和失效概率。

Listing 6: solve_q5.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4 import common
5
6 import numpy as np
7 import joblib as jb
8 from scipy.stats import qmc
9 from SALib.sample import sobol as sobol_sample
10 from SALib.analyze import sobol as sobol_analyze
11
12 import solve_q3 as q3
13
14 res = jb.load(common.DATA_DIR / "gpr.pkl")
15 scaler = res["_scaler"]
16 outputs = ["Rth", "dP", "dT"]
17 gprs = {col: res[col]["gpr"] for col in outputs}
18
19 EPS = 0.05 # 综合扰动幅度 +-5%(eq:perturb)
20 N_MC = 8000 # LHS蒙特卡洛样本数, >=5000 (FILL_CHECKLIST 5.1要求)
21
22 # 结构参数标称设计点: 用问题三的综合最优方案  $\mathbf{X}^*$  作为标称点 (其余分析都在此点展开)
23 q3_res = jb.load(common.DATA_DIR / "q3_result.pkl")
24 X0 = np.array(q3_res["x_star"]) # [alpha0, beta0, n0]
25
26 # 工况参数标称点 (附件2采集时的固定工况, 04-models.tex式 (eq:perturb) 前文)
27 MDOT0, TIN0, QV0 = 1.0, 293.0, 5e9 # g/s, K, W/m^3 (仅用于表示相对扰动, 不代入GPR)
28
29
30 def predict(X):
31     return q3.predict(np.atleast_2d(X))
32
33
34 # ----- 5.1 结构参数层 (严格, GPR直接支撑) -----
35
36 # Sobol/LSA/LHS用的结构参数域: 取"有针肋"模式的连续box (x3当连续处理,
37 # 是GSA里对整数变量的常规近似, 评估时不取整, 因为这里研究的是"加工误差"
38 # 连续扰动, 不是重新做整数设计搜索)
39 SOBOL_PROBLEM = {
40     "num_vars": 3,
41     "names": ["x1", "x2", "x3"],
```

```

42     "bounds": [list(q3.X1_FINNED_BOUNDS), [3.00, 4.50], [2, 10]],
43 }
44
45
46 def sobol_indices(N=1024):
47     """Sobol一阶主效应Si与总效应STi,对三个结构参数x1,x2,x3、三个输出分别给出"""
48     param_values = sobol_sample.sample(SOBOL_PROBLEM, N)
49     result = {}
50     for col in outputs:
51         X_std = scaler.transform(param_values)
52         Y = gprs[col].predict(X_std)
53         Si = sobol_analyze.analyze(SOBOL_PROBLEM, Y, print_to_console=False)
54         result[col] = {"S1": Si["S1"], "ST": Si["ST"]}
55     return result
56
57
58 def lsa_structural(X0=X0, h_rel=1e-3):
59     """局部归一化敏感性系数 Sijlocal = dGj/dxi * xi0/yj0,中心差分(eq:lsa)"""
60     F0 = predict(X0)[0]
61     S = np.zeros((3, 3)) # 行=x1,x2,x3 列=Rth,dP,dT
62     for i in range(3):
63         h = max(abs(X0[i]) * h_rel, 1e-4)
64         Xp, Xm = X0.copy(), X0.copy()
65         Xp[i] += h
66         Xm[i] -= h
67         Fp = predict(Xp)[0]
68         Fm = predict(Xm)[0]
69         dF = (Fp - Fm) / (2 * h)
70         S[i] = dF * X0[i] / F0
71     return S # S[i,j]
72
73
74 def lhs_montecarlo(X0=X0, eps=EPS, N=N_MC, seed=676767):
75     """结构参数层LHS蒙特卡洛不确定性传播(eq:perturb, sigma_zi=eps*zi0/3)"""
76     sigma = eps * np.abs(X0) / 3
77     sampler = qmc.LatinHypercube(d=3, seed=seed)
78     U = sampler.random(N) # (N,3) in [0,1)
79     from scipy.stats import norm
80     Z = norm.ppf(U) # 标准正态分位数
81     samples = X0 + Z * sigma
82     F = predict(samples) # (N,3): Rth,dP,dT
83     F0 = predict(X0)[0]
84     cv = F.std(axis=0, ddof=1) / F.mean(axis=0)
85     p95 = np.quantile(F, 0.95, axis=0)
86     y_limit = 1.10 * F0
87     p_fail = (F > y_limit).mean(axis=0)
88     return {"F0": F0, "cv": cv, "p95": p95, "y_limit": y_limit, "p_fail": p_fail, "F_samples": F}
89
90
91 # ----- 5.2 工况参数层(近似,线性化,eq:op-linear) -----
92 #
93 # GPR从未在mdot/T_in/q_v三个维度上训练过(附件2数据全部在固定标称工况下采集),
94 # 所以论文里"Sijlocal由代理模型梯度有限差分近似"这句话对工况参数层literally
95 # 是算不出来的——这是main分支论文文本里遗留的一处表述和可执行性之间的缺口。
96 # 这里改用基于05-models.tex 5.5.1节(1)已经建立的无量纲化结果(Re,Pr主导)
97 # 做一阶物理标度估算,不是GPR梯度,推导依据见下方注释,结果需要论文手确认

```

```

98 # 措辞是否要改成"基于机理标度关系近似"而不是"代理模型梯度"。
99 #
100 # 推导:
101 # - mdot: 固定几何下  $Re \propto \dot{m}$ 。层流微通道热入口段常用关联式  $Nu \sim Re^{1/3} Pr^{1/3}$ 
102 # (Leveque型),  $h \propto Nu$ , 故  $Rth \propto 1/h \propto \dot{m}^{-1/3} \Rightarrow S = -1/3$ ;  $dT^*$  的驱动机制与
103 # 对流换热同源, 近似取同一量级  $S = -1/3$ 。
104 # 压降(层流Poiseuille,  $f = 64/Re \propto 1/\dot{m}$ , 原始  $\Delta P \propto f \dot{m}^2 \propto \dot{m}$ ;
105 # 无量纲化分母  $\rho U^2 \propto \dot{m}^2$ , 故  $dP^* \propto \dot{m}^{-1} \Rightarrow S = -1$ 。
106 # -  $T_{in}$ : 无量纲温度以  $\Delta T_{ref}$  (与  $T_{in}$  相关的参考温差) 归一化, 一阶展开下
107 #  $T_{in}$  的直接贡献被归一化过程抵消, 只剩物性随温度变化的二阶效应, 取  $S$  约等于 0。
108 # -  $q_v$ : 对流-传导线性问题中,  $\Delta T_{ref}$  通常正比于  $q_v$ ,  $Rth^* = \Delta T_{actual} / \Delta T_{ref}$ 、
109 #  $dT^*$  同理, 分子分母同比例缩放, 一阶抵消,  $S$  约等于 0;  $dP^*$  是纯流体力学量,
110 # 与发热功率解耦,  $S = 0$ 。
111 OP_SENSITIVITY = {
112     #      Rth    dP    dT
113     "mdot": {"Rth": -1 / 3, "dP": -1.0, "dT": -1 / 3},
114     "Tin": {"Rth": 0.0, "dP": 0.0, "dT": 0.0},
115     "qv": {"Rth": 0.0, "dP": 0.0, "dT": 0.0},
116 }
117
118
119 def op_linear_impact(eps=EPS):
120     """工况参数+-eps扰动下,按eq:op-linear线性叠加估算三项指标的相对变化幅度"""
121     impact = {}
122     for col in outputs:
123         s = OP_SENSITIVITY["mdot"][col] * eps + OP_SENSITIVITY["Tin"][col] * eps + OP_SENSITIVITY["qv"
124             ↪ ] [col] * eps
125         impact[col] = s # 三个工况参数同向拉满+eps的最坏情形线性叠加(相对变化量)
126     return impact
127
128 # ----- 5.3 验证闭环对比表(tab:q5-compare) -----
129
130 def verification_closure(eps=EPS, N=N_MC):
131     """X*(问题三)与X_robust(问题四)分别过结构层MC + 工况层线性叠加,对比CV/Pfail"""
132     q4_res = jb.load(common.DATA_DIR / "q4_result.pkl")
133     X_star = np.array(q3_res["x_star"])
134     X_robust = np.array(q4_res["x_robust"])
135
136     op_impact = op_linear_impact(eps) # {col: relative shift}
137     # 工况层线性叠加是确定性的均值平移,不改变结构MC样本的绝对标准差,
138     # 只平移均值:合成CV_j = std(结构MC样本)/(mean(结构MC样本)*(1+工况线性偏移))
139
140     rows = {}
141     for name, X in [("X_star", X_star), ("X_robust", X_robust)]:
142         mc = lhs_montecarlo(X0=X, eps=eps, N=N)
143         op_shift = np.array([op_impact[col] for col in outputs])
144         mean_combined = mc["F0"] * (1 + op_shift)
145         std_struct = mc["cv"] * mc["F0"]
146         cv_combined = std_struct / mean_combined
147         y_limit = 1.10 * mc["F0"]
148         p_fail_combined = (mc["F_samples"] * (1 + op_shift) > y_limit).mean(axis=0)
149         rows[name] = {
150             "X": X, "F0": mc["F0"],
151             "cv": cv_combined, "p_fail": p_fail_combined,
152             "meets_cv": bool((cv_combined < 0.03).all()),

```



```

153     "meets_pfail": bool((p_fail_combined < 1e-3).all()),
154 }
155 return rows
156
157
158 def solve():
159     print(f"标称点 X0(=Q3综合最优X*) = alpha={X0[0]:.4f} beta={X0[1]:.2f} n={X0[2]:.4f}")
160
161     print("\n== 5.1(1) 局部归一化敏感性系数 S_ij^local(结构参数,标称点X0) ==")
162     S_lsa = lsa_structural()
163     for i, name in enumerate(["x1(alpha)", "x2(beta)", "x3(n)"]):
164         print(f" {name}: Rth={S_lsa[i,0]:+.4f} dP={S_lsa[i,1]:+.4f} dT={S_lsa[i,2]:+.4f}")
165
166     print("\n== 5.1(2) Sobol全局敏感性指数(结构参数x1,x2,x3) ==")
167     sobol_res = sobol_indices(N=1024)
168     for col in outputs:
169         S1 = sobol_res[col]["S1"]
170         ST = sobol_res[col]["ST"]
171         print(f" {col}: S1(x1,x2,x3)={np.round(S1,4)} ST(x1,x2,x3)={np.round(ST,4)}")
172
173     print("\n== 5.1(3) LHS蒙特卡洛不确定性传播(N={N_MC}, eps={EPS*100:.0f}% ,结构层X0=X*) ==")
174     mc = lhs_montecarlo()
175     for j, col in enumerate(outputs):
176         print(f" {col}: y0={mc['F0'][j]:.4f} CV={mc['cv'][j]*100:.4f}% "
177               f"P95={mc['p95'][j]:.4f} y_limit={mc['y_limit'][j]:.4f} Pfail={mc['p_fail'][j]*100:.4f}% "
178               ↪ )
179
180     print("\n== 5.2 工况参数层线性化影响(近似,基于机理标度关系,非GPR梯度——详见代码注释) ==")
181     op_impact = op_linear_impact()
182     for col in outputs:
183         print(f" {col}: +{EPS*100:.0f}%综合工况扰动 -> 相对变化 {op_impact[col]*100:+.2f}%")
184
185     print("\n== 5.3 验证闭环对比表(tab:q5-compare) ==")
186     closure = verification_closure()
187     for name, row in closure.items():
188         cv_pct = row["cv"] * 100
189         pfail_max = row["p_fail"].max()
190         print(f" {name}: X={np.round(row['X'],4)}")
191         print(f" CV1={cv_pct[0]:.4f}% CV2={cv_pct[1]:.4f}% CV3={cv_pct[2]:.4f}% "
192               f"Pfail(max over 3)={pfail_max*100:.4f}% "
193               f"CV<3%达标={row['meets_cv']} Pfail~=0达标={row['meets_pfail']}")
194
195     # ---- 5.4/6.2 最敏感参数结论(取Sobol总效应ST在3个输出上的最大值判定) ----
196     ST_matrix = np.array([sobol_res[col]["ST"] for col in outputs]) # (3out, 3param)
197     flat_idx = np.unravel_index(np.argmax(ST_matrix), ST_matrix.shape)
198     most_sensitive_param = ["x1(alpha, 针肋宽度比)", "x2(beta, 歧管深高比)", "x3(n, 针肋排数)"][flat_idx[1]]
199     most_sensitive_output = outputs[flat_idx[0]]
200     print(f"\n== 5.4/6.2 结论 ==")
201     print(f"最敏感参数-指标组合: {most_sensitive_output} 对 {most_sensitive_param} 的扰动最敏感 "
202           f"(ST={ST_matrix[flat_idx]:.4f})")
203     strong_params = []
204     for pi, pname in enumerate(["x1", "x2", "x3"]):
205         if (ST_matrix[:, pi] > 0.2).any():
206             strong_params.append(pname)
207     print(f"强敏感参数(存在某指标ST>0.2): {strong_params if strong_params else '无'}")

```

```

208 out = {
209     "X0": X0, "S_lsa": S_lsa, "sobol": sobol_res, "mc": mc,
210     "op_impact": op_impact, "closure": closure,
211     "most_sensitive_param": most_sensitive_param,
212     "most_sensitive_output": most_sensitive_output,
213     "strong_params": strong_params,
214 }
215 jb.dump(out, common.DATA_DIR / "q5_result.pkl")
216 return out
217
218
219 if __name__ == "__main__":
220     solve()

```

1.7 fig_q1.py —问题—配图

绘制 $\beta = 3.75$ 固定时不同 n 取值下 α - R_{th}^* 曲线族 (标注 α^* 拐点) 和 α - ΔP^* 曲线族。

Listing 7: fig_q1.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from matplotlib import cm
8 import common
9 import joblib as jb
10
11 res = jb.load(common.DATA_DIR / "gpr.pkl")
12 scaler = res["_scaler"]
13 gpr_rth = res["Rth"]["gpr"]
14 gpr_dp = res["dP"]["gpr"]
15
16 ALPHAS = np.linspace(0.10, 0.30, 81)
17 BETA_FIXED = 3.75 # 歧管深高比取值中点的代表值
18 N_LEVELS = [2, 4, 6, 8, 10]
19
20 # n 为有序变量,用 viridis(感知均匀、色盲安全)的 5 档渐变色编码排数递增
21 N_COLORS = [cm.viridis(0.15 + 0.7 * i / (len(N_LEVELS) - 1)) for i in range(len(N_LEVELS))]
22
23 fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.8))
24
25 # ----- 左图: alpha-Rth 曲线族,标注拐点 -----
26 ax = axes[0]
27 avg_curve = np.zeros_like(ALPHAS)
28 for n, c in zip(N_LEVELS, N_COLORS):
29     X = np.column_stack([ALPHAS, np.full_like(ALPHAS, BETA_FIXED), np.full_like(ALPHAS, n)])
30     y = gpr_rth.predict(scaler.transform(X))
31     avg_curve += y
32     ax.plot(ALPHAS, y, color=c, lw=1.8, label=f"$n={n}$")
33 avg_curve /= len(N_LEVELS)
34
35 # 拐点:对曲线族平均求 argmin 并标注(该  $\beta$  处的拐点区域,与论文全局交叉验证值  $\alpha^* \approx 0.2$  一致)
36 i_star = np.argmin(avg_curve)
37 alpha_star = ALPHAS[i_star]
38 ax.axvline(alpha_star, linestyle="--", color="gray", lw=1.2)
39 ax.annotate(
40     f"拐点区域  $\alpha \approx \alpha_{star:.2f}$ ",
41     xy=(alpha_star, avg_curve[i_star]), xycoords="data",
42     xytext=(alpha_star + 0.028, avg_curve[i_star] + 0.006), textcoords="data",
43     arrowprops=dict(arrowstyle="->", color="gray", lw=1.0),
44     fontsize=10, color="dimgray",
45 )
46 ax.set_xlabel(r"针肋宽度比  $\alpha$ ")
47 ax.set_ylabel(r"无量纲热阻  $R_{th}^*$ ")
48 ax.set_title(f"(a)  $\alpha$ - $R_{th}^*$  曲线族( $\beta={BETA\_FIXED}$ )")
49 ax.legend(fontsize=8, ncol=5, loc="upper center", frameon=False,
50         bbox_to_anchor=(0.5, -0.13))
```

```

51 ax.set_ylim(0.72, 0.775)
52
53 # ----- 右图: alpha-dP 曲线族 -----
54 ax = axes[1]
55 for n, c in zip(N_LEVELS, N_COLORS):
56     X = np.column_stack([ALPHAS, np.full_like(ALPHAS, BETA_FIXED), np.full_like(ALPHAS, n)])
57     y = gpr_dp.predict(scaler.transform(X))
58     ax.plot(ALPHAS, y, color=c, lw=1.8, label=f"$n={n}$")
59 ax.set_xlabel(r"针肋宽度比 $\alpha$")
60 ax.set_ylabel(r"无量纲压降 $\Delta P^*$")
61 ax.set_title(f"(b) $\alpha$-$\Delta P^*$ 曲线族 ($\beta={BETA\_FIXED}$)")
62 ax.legend(fontsize=8, ncol=5, loc="upper center", frameon=False,
63          bbox_to_anchor=(0.5, -0.13))
64
65 fig.tight_layout()
66 common.savefig("fig_q1.png")

```

1.8 fig_q2.py —问题二配图

三个输出指标 (R_{th}^* , ΔP^* , ΔT^*) 的 GPR 预测值与真实值散点对比图, 附 $y = x$ 参考线。

Listing 8: fig_q2.py

```

1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import common
8 import joblib as jb
9
10 res = jb.load(common.DATA_DIR / "gpr.pkl")
11 outputs = [k for k in res.keys() if not k.startswith("_")]
12
13 # 指标 → 颜色语义: Rth=蓝 dP=橙 dT=绿(Okabe-Ito 色盲安全)
14 OUT_COLOR = {
15     "Rth": common.COLOR_RTH,
16     "dP": common.COLOR_DP,
17     "dT": common.COLOR_DT,
18 }
19 OUT_LABEL = {
20     "Rth": r"$R_{th}^*$",
21     "dP": r"$\Delta P^*$",
22     "dT": r"$\Delta T^*$",
23 }
24
25 fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.3))
26
27 for col, ax in zip(outputs, axes):
28     y_true = res[col]["y_true"]
29     y_pred = res[col]["y_pred"]

```

```

30 r2 = res[col]["r2"]
31 rmse = res[col]["rmse"]
32
33 ax.scatter(y_pred, y_true, s=26, alpha=0.75, edgecolors="white",
34           linewidths=0.5, color=OUT_COLOR[col], zorder=3)
35 ax.axline((0, 0), slope=1, linestyle="--", color="gray", lw=1.0, zorder=2)
36
37 # 拟合精度标注(左上角,数据点集中在右上,不冲突)
38 ax.text(0.04, 0.94,
39        f"$R^2={r2:.4f}$\nRMSE$={rmse:.2e}$",
40        transform=ax.transAxes, va="top", ha="left",
41        fontsize=9, bbox=dict(boxstyle="round,pad=0.3",
42                               fc="white", ec="lightgray", alpha=0.85))
43
44 ax.set_title(f"{OUT_LABEL[col]}")
45 ax.set_xlabel("GPR 预测值")
46 ax.set_ylabel("真实值(CFD)")
47 ax.set_aspect("equal")
48 lo = min(y_true.min(), y_pred.min())
49 hi = max(y_true.max(), y_pred.max())
50 pad = (hi - lo) * 0.12
51 ax.set_xlim(lo - pad, hi + pad)
52 ax.set_ylim(lo - pad, hi + pad)
53 # 只保留少数刻度,避免数字堆叠
54 ax.ticklabel_format(style="plain", useOffset=False)
55 ax.set_xticks(np.round(np.linspace(lo, hi, 4), 3))
56 ax.set_yticks(np.round(np.linspace(lo, hi, 4), 3))
57
58 fig.tight_layout()
59 common.savefig("fig_q2_true_pred.png")

```

1.9 fig_sensitivity.py —问题五配图

四合一敏感性分析图板: (a) Sobol 总效应指数柱状图; (b) 局部归一化敏感性系数热力图 (RdBu_r 发散色); (c) $\pm 5\%$ 扰动下变异系数对比 (含工况线性叠加); (d) ΔT 的一阶主效应与总效应对比。

Listing 9: fig_sensitivity.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import common
8 import joblib as jb
9
10 res = jb.load(common.DATA_DIR / "q5_result.pkl")
11 sobol = res["sobol"]
12 S_lsa = res["S_lsa"]
13 closure = res["closure"]
14
15 outputs = ["Rth", "dP", "dT"]
16 param_names = [r"$x_1$ ($\alpha$)", r"$x_2$ ($\beta$)", r"$x_3$ ($\eta$)"]
17 param_short = ["x1", "x2", "x3"]
18 out_labels = [r"$R_{th}^*$", r"$\Delta P^*$", r"$\Delta T^*$"]
19
20 # 指标 → 颜色语义(与 fig_q2 一致): Rth=蓝 dP=橙 dT=绿
21 OUT_COLOR = [common.COLOR_RTH, common.COLOR_DP, common.COLOR_DT]
22 OUT_EDGE = ["#00517A", "#8A3A00", "#006B52"] # 同色系深色描边, 柱状更有层次
23
24 fig, axes = plt.subplots(2, 2, figsize=(13, 8.6))
25
26 # ----- (a) Sobol 总效应指数: 分组柱状 -----
27 ax = axes[0, 0]
28 x = np.arange(len(param_names))
29 width = 0.25
30 for j, col in enumerate(outputs):
31     ST = sobol[col]["ST"]
32     bars = ax.bar(x + j * width, ST, width, label=out_labels[j],
33                  color=OUT_COLOR[j], edgecolor=OUT_EDGE[j], lw=0.6, zorder=3)
34     for b in bars: # 柱顶数值标注
35         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.012,
36                f"{b.get_height():.2f}", ha="center", va="bottom",
37                fontsize=7.5, color="dimgray")
38 ax.axhline(0.2, linestyle="--", color="gray", lw=1.0, zorder=2)
39 ax.text(2.62, 0.215, "ST = 0.2", fontsize=8, color="gray", ha="right")
40 ax.set_xticks(x + width)
41 ax.set_xticklabels(param_names)
42 ax.set_ylabel("Sobol 总效应指数 $S_{Ti}$")
43 ax.set_ylim(0, 0.85)
44 ax.set_title("(a) Sobol 全局敏感性总效应指数")
45 ax.legend(fontsize=8, frameon=False, loc="upper left")
46
```

```

47 # ----- (b) 局部归一化敏感性系数 LSA:热力图(原折线违反"离散点不连折线") -----
48 ax = axes[0, 1]
49 im = ax.imshow(S_lsa, cmap="RdBu_r", vmin=-0.9, vmax=0.9, aspect="auto")
50 ax.set_xticks(range(3))
51 ax.set_xticklabels(out_labels)
52 ax.set_yticks(range(3))
53 ax.set_yticklabels(param_names)
54 # 单元格数值标注:正负号+两位小数,深底用白字
55 for i in range(3):
56     for j in range(3):
57         v = S_lsa[i, j]
58         text_color = "white" if abs(v) > 0.55 else "black"
59         ax.text(j, i, f"{v:+.2f}", ha="center", va="center",
60               fontsize=9.5, color=text_color)
61 ax.set_title("(b) 局部归一化敏感性系数  $S_{ij}^{\text{local}}$  (标称点  $X^*$ )")
62 cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
63 cbar.set_label(r" $S_{ij}^{\text{local}} = \partial G_j / \partial x_i \cdot x_i^0 / y_j^0$ ",
64               fontsize=9)
65
66 # ----- (c)  $\pm 5\%$  扰动变异系数对比:分组柱状 + 阈值线 -----
67 ax = axes[1, 0]
68 X_star_cv = closure["X_star"]["cv"] * 100
69 X_robust_cv = closure["X_robust"]["cv"] * 100
70 x = np.arange(len(outputs))
71 width = 0.35
72 b1 = ax.bar(x - width / 2, X_star_cv, width, label=r"综合最优  $X^*$ ",
73            color=common.COLOR_RTH, edgecolor="#00517A", lw=0.6, zorder=3)
74 b2 = ax.bar(x + width / 2, X_robust_cv, width, label=r"鲁棒方案  $X_{\text{robust}}$ ",
75            color=common.COLOR_DP, edgecolor="#8A3A00", lw=0.6, zorder=3)
76 for bars in (b1, b2):
77     for b in bars:
78         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.03,
79               f"{b.get_height():.2f}", ha="center", va="bottom",
80               fontsize=8, color="dimgray")
81 ax.axhline(3.0, linestyle="--", color="gray", lw=1.0, zorder=2)
82 ax.text(2.4, 3.08, "CV = 3% 阈值", fontsize=8, color="gray", ha="right")
83 ax.set_xticks(x)
84 ax.set_xticklabels(out_labels)
85 ax.set_ylabel("变异系数 CV (%)")
86 ax.set_ylim(0, 3.6)
87 ax.set_title("(c)  $\pm 5\%$  扰动下变异系数对比(含工况线性叠加)")
88 ax.legend(fontsize=8, frameon=False, loc="upper left")
89
90 # ----- (d)  $\Delta T$  的一阶 vs 总效应:分组柱状 -----
91 ax = axes[1, 1]
92 col = "dT"
93 S1 = sobol[col]["S1"]
94 ST = sobol[col]["ST"]
95 x = np.arange(len(param_names))
96 b1 = ax.bar(x - width / 2, S1, width, label=r"一阶主效应  $S_{i1}$ ",
97            color="#56B4E9", edgecolor="#1E7DB8", lw=0.6, zorder=3)
98 b2 = ax.bar(x + width / 2, ST, width, label=r"总效应  $S_{Ti}$ ",
99            color=common.COLOR_RTH, edgecolor="#00517A", lw=0.6, zorder=3)
100 for bars in (b1, b2):
101     for b in bars:
102         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.012,

```

```

103         f"{b.get_height():.2f}", ha="center", va="bottom",
104         fontsize=7.5, color="dimgray")
105 ax.set_xticks(x)
106 ax.set_xticklabels(param_names)
107 ax.set_ylabel("Sobol 指数")
108 ax.set_ylim(0, 0.85)
109 ax.set_title(r"(d)  $\Delta T^*$  的一阶主效应与总效应对比")
110 ax.legend(fontsize=8, frameon=False, loc="upper left")
111
112 fig.tight_layout()
113 common.savefig("fig_sensitivity.png")

```


1.10 fig_pareto.py —问题三配图

Pareto 前沿的三目标两两投影散点图, 点色按理想点距离 D 着色 (viridis), ★ 标记综合最优 $\mathbf{F}^*$, 支撑”三目标冲突、不存在单一最优”的论证。

Listing 10: fig_pareto.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from matplotlib import cm
8 import common
9 import joblib as jb
10
11 res = jb.load(common.DATA_DIR / "q3_result.pkl")
12 F_front = res["F_front"]
13 f_min = res["f_min"]
14 f_max = res["f_max"]
15 F_star = res["F_star"]
16
17 # 每个前沿解的归一化理想点距离 (颜色编码: 好解盆地)
18 F_norm = (F_front - f_min) / (f_max - f_min)
19 D = np.sqrt(np.sum(F_norm ** 2, axis=1) / 3)
20
21 PAIRS = [
22     (0, 1, r"$R_{th}^{*}$", r"$\Delta P^{*}$", "(a)  $R_{th}^{*}$ - $\Delta P^{*}$  投影"),
23     (0, 2, r"$R_{th}^{*}$", r"$\Delta T^{*}$", "(b)  $R_{th}^{*}$ - $\Delta T^{*}$  投影"),
24     (1, 2, r"$\Delta P^{*}$", r"$\Delta T^{*}$", "(c)  $\Delta P^{*}$ - $\Delta T^{*}$  投影"),
25 ]
26
27 fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
28 sc = None
29 for ax, (i, j, xlabel, ylabel, title) in zip(axes, PAIRS):
30     sc = ax.scatter(F_front[:, i], F_front[:, j], c=D, cmap="viridis",
31                     s=10, alpha=0.55, linewidths=0, zorder=2)
32     ax.scatter([F_star[i]], [F_star[j]], marker="*", s=260,
33               color=common.COLOR_RTH, edgecolors="white",
34               linewidths=1.0, zorder=4)
35     ax.annotate(
36         r"$\mathbf{F}^{*}$",
37         xy=(F_star[i], F_star[j]), xycoords="data",
38         xytext=(8, 12), textcoords="offset points",
39         fontsize=11, color=common.COLOR_RTH,
40         arrowprops=dict(arrowstyle="->", color=common.COLOR_RTH, lw=1.0),
41     )
42     ax.set_xlabel(xlabel)
43     ax.set_ylabel(ylabel)
44     ax.set_title(title)
45
46 cbar = fig.colorbar(sc, ax=axes, fraction=0.025, pad=0.02)
47 cbar.set_label("理想点距离  $D$ ", fontsize=9)
48 cbar.set_ticks([D.min(), (D.min() + D.max()) / 2, D.max()])
```

```
49 cbar.set_ticklabels([f"{D.min():.2f}(优)", f"{(D.min()+D.max())/2:.2f}", f"{D.max():.2f}(劣)"])\n50\n51 fig.tight_layout()\n52 common.savefig("fig_pareto.png")
```

1.11 fig_weight_sweep.py —问题四配图

全偏好空间扫描图: 权重单纯形 (重心坐标投影) 上 31 个采样点, (a) 按最优排数 n^* 分区着色, 标注典型场景点与等权点; (b) 按综合距离 $D^*(\mathbf{W})$ 连续着色, 展示偏好空间内最优解的光滑变化。

Listing 11: fig_weight_sweep.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from matplotlib import cm
8 import common
9 import joblib as jb
10
11 res = jb.load(common.DATA_DIR / "q4_result.pkl")
12 all_w = res["all_w"] # (31, 3) 权重单纯形采样点
13 X_star = res["X_star"] # (31, 3) 各权重下的最优解
14 D_star = res["D_star"] # (31,)
15 scenario_w = res["scenario_w"]
16 scenario_names = res["scenario_names"]
17
18 # 重心坐标 → 等边三角形平面投影(无额外依赖)
19 def simplex_xy(w):
20     return w[:, 0] + 0.5 * w[:, 2], w[:, 2] * np.sqrt(3) / 2
21
22 # 三角形三个顶点(w1=1, w2=1, w3=1)与边线
23 def draw_triangle(ax):
24     verts_w = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]], dtype=float)
25     xs, ys = simplex_xy(verts_w)
26     for k in range(3):
27         ax.plot([xs[k], xs[(k + 1) % 3]], [ys[k], ys[(k + 1) % 3]],
28                 color="gray", lw=1.2, zorder=1)
29     # 顶点标签(放在三角形外侧)
30     offset = [(0.0, -0.075), (0.0, -0.075), (0.13, 0.02)]
31     labels = [r"$w_1=1$(散热)", r"$w_2=1$(泵功)", r"$w_3=1$(均匀性)"]
32     for (x, y), lab, (dx, dy) in zip(zip(xs, ys), labels, offset):
33         ax.text(x + dx, y + dy, lab, ha="center", va="top", fontsize=9, color="dimgray")
34
35 px, py = simplex_xy(all_w)
36 n_opt = X_star[:, 2] # 各权重下的最优排数
37
38 fig, axes = plt.subplots(1, 2, figsize=(12.5, 5.2))
39
40 # ----- (a) 偏好空间内最优排数的分区结构 -----
41 ax = axes[0]
42 draw_triangle(ax)
43 levels = sorted(set(n_opt.astype(int)))
44 cmap = cm.viridis
45 norm_n = plt.Normalize(min(levels) - 0.5, max(levels) + 0.5)
46 sc = ax.scatter(px, py, c=n_opt, cmap=cmap, norm=norm_n, s=95,
```

```

47     edgecolors="white", linewidths=0.8, zorder=3)
48 # 三个典型场景点 + 等权点
49 sx, sy = simplex_xy(scenario_w)
50 ax.scatter(sx, sy, marker="s", s=110, facecolors="none",
51           edgecolors=common.COLOR_RTH, linewidths=1.6, zorder=4)
52 eq_x, eq_y = simplex_xy(np.array([[1/3, 1/3, 1/3]]))
53 ax.scatter(eq_x, eq_y, marker="D", s=90, facecolors="none",
54           edgecolors=common.COLOR_DP, linewidths=1.6, zorder=4)
55 ax.text(eq_x[0] + 0.02, eq_y[0] - 0.055, "等权点", fontsize=8,
56        color=common.COLOR_DP, ha="left")
57 for (x, y), name in zip(zip(sx, sy), scenario_names):
58     ax.text(x + 0.015, y, name, fontsize=7.5, color=common.COLOR_RTH, ha="left")
59 cbar = fig.colorbar(sc, ax=ax, fraction=0.046, pad=0.04,
60                   ticks=levels)
61 cbar.set_label("最优针肋排数  $n^*$ ", fontsize=9)
62 ax.set_aspect("equal")
63 ax.axis("off")
64 ax.set_title("(a) 偏好空间内最优排数分区")
65
66 # ----- (b)  $D^*(W)$  在偏好空间的光滑变化 -----
67 ax = axes[1]
68 draw_triangle(ax)
69 D_zero = np.where(D_star < 1e-6, np.nan, D_star) # 顶点处  $D^*=0$  (单目标退化), 置 NaN 保持色标有意义
70 sc = ax.scatter(px, py, c=D_zero, cmap="viridis", s=95,
71               edgecolors="white", linewidths=0.8, zorder=3)
72 ax.scatter(sx, sy, marker="s", s=110, facecolors="none",
73           edgecolors=common.COLOR_RTH, linewidths=1.6, zorder=4)
74 ax.scatter(eq_x, eq_y, marker="D", s=90, facecolors="none",
75           edgecolors=common.COLOR_DP, linewidths=1.6, zorder=4)
76 cbar = fig.colorbar(sc, ax=ax, fraction=0.046, pad=0.04)
77 cbar.set_label(r"综合距离  $D^*(\mathbf{W})$ ", fontsize=9)
78 ax.set_aspect("equal")
79 ax.axis("off")
80 ax.set_title("(b)  $D^*(\mathbf{W})$  在偏好空间的光滑变化")
81
82 fig.tight_layout()
83 common.savefig("fig_weight_sweep.png")

```

1.12 fig_mc_dist.py —问题五配图

$\pm 5\%$ 扰动下三项指标的蒙特卡洛响应分布直方图 + KDE, 标注标称值 y_0 、95% 分位数 y_{P95} 与 CV/P_{fail} 数值, 越界阈值 $1.10y_0$ 在分布尾部之外用右缘箭头示意。

Listing 12: fig_mc_dist.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from scipy.stats import gaussian_kde
8 import common
9 import joblib as jb
10
11 res = jb.load(common.DATA_DIR / "q5_result.pkl")
12 mc = res["mc"]
13 F_samples = mc["F_samples"] # (8000, 3)
14 F0 = mc["F0"] # 标称值
15 p95 = mc["p95"] # 95% 分位数
16 y_limit = mc["y_limit"] # 越界阈值 1.10*F0
17 cv = mc["cv"]
18 p_fail = mc["p_fail"]
19
20 outputs = ["Rth", "dP", "dT"]
21 OUT_LABEL = [r"$R_{th}^{*}$", r"$\Delta P^{*}$", r"$\Delta T^{*}$"]
22 OUT_COLOR = [common.COLOR_RTH, common.COLOR_DP, common.COLOR_DT]
23
24 fig, axes = plt.subplots(1, 3, figsize=(14, 4.2))
25
26 for j, (ax, col, lab, color) in enumerate(zip(axes, outputs, OUT_LABEL, OUT_COLOR)):
27     x = F_samples[:, j]
28     # 直方图 + KDE 密度曲线
29     n_bins = 60
30     ax.hist(x, bins=n_bins, density=True, color=color, alpha=0.45,
31            edgecolor="white", linewidth=0.4, zorder=2)
32     kde = gaussian_kde(x)
33     xs = np.linspace(x.min(), x.max(), 300)
34     ax.plot(xs, kde(xs), color=color, lw=2.0, zorder=3)
35
36     # 竖线: 标称值 / P95 (越界阈值 1.10y0 远在分布尾部之外, 面板右侧用箭头示意)
37     ax.axvline(F0[j], color="black", lw=1.2, zorder=4)
38     ax.axvline(p95[j], color="gray", linestyle=":", lw=1.4, zorder=4)
39
40     # 数值标注
41     ax.text(0.03, 0.95, f"CV = {cv[j]*100:.2f}%\nP_{fail} = {p_fail[j]*100:.2f}%",
42            transform=ax.transAxes, va="top", ha="left", fontsize=9,
43            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="lightgray", alpha=0.85))
44     # 越界阈值箭头提示 (线在可视范围外, 用右缘箭头表示)
45     ax.annotate(
46         r"越界阈值 $1.10y_0 \rightarrow$",
47         xy=(0.995, 0.60), xycoords="axes fraction",
48         xytext=(0.79, 0.97), textcoords="axes fraction",
```

```

49     ha="right", va="top", fontsize=8, color=common.COLOR_DP,
50     arrowprops=dict(arrowstyle="->", color=common.COLOR_DP, lw=1.0),
51 )
52 # 两条线的图例(右上)
53 from matplotlib.lines import Line2D
54 handles = [
55     Line2D([0], [0], color="black", lw=1.2, label=r"标称值  $y_0$ "),
56     Line2D([0], [0], color="gray", lw=1.4, linestyle=":", label=" $y_{P95}$ "),
57 ]
58 ax.legend(handles=handles, fontsize=7.5, frameon=False, loc="upper right")
59
60 ax.set_xlabel(lab)
61 ax.set_ylabel("概率密度")
62 ax.set_title(f"{lab}( $\pm 5\%$  扰动,  $N=8000$ )")
63 ax.set_ylim(bottom=0)
64
65 fig.tight_layout()
66 common.savefig("fig_mc_dist.png")

```

1.13 fig_regret.py —问题四配图

候选池 2000 个方案的最大后悔值分布直方图, 标注鲁棒方案 R_{max} 、容许上限 ϵ 与等权综合最优 X^* 的最大后悔值 (后者在脚本内按与候选池同口径重新计算)。

Listing 13: fig_regret.py

```
1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parent))
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import common
8 import joblib as jb
9 import solve_q3 as q3 # 复用 predict/normalize
10
11 res = jb.load(common.DATA_DIR / "q4_result.pkl")
12 max_regret = res["max_regret"] # (2000,) 候选池各方案的最大后悔值
13 R_max = res["R_max"] # minimax 最优后悔值(鲁棒方案)
14 epsilon = res["epsilon"] # 容许上限
15 all_w = res["all_w"]
16 D_star_grid = res["D_star"]
17 f_min, f_max = res["f_min"], res["f_max"]
18 x_star = np.array(jb.load(common.DATA_DIR / "q3_result.pkl")["x_star"]) # 等权综合最优
19
20 # 计算等权综合最优  $X^*$  在全偏好空间的最大后悔值(与候选池同口径)
21 F_xs = q3.predict(np.atleast_2d(x_star))[0]
22 F_norm = (F_xs - f_min) / (f_max - f_min)
23 regret_xs = np.sqrt(np.sum(all_w * F_norm ** 2, axis=1)) - D_star_grid
24 regret_xs_max = regret_xs.max()
25
26 fig, ax = plt.subplots(figsize=(8.5, 4.5))
27
28 # 直方图主区(x 从 0.20 起, 让  $\epsilon$  与直方图的量级差距直观可见)
29 ax.hist(max_regret, bins=45, color="#B8C4D0", edgecolor="white",
30         linewidth=0.4, zorder=2)
31 ax.axvline(R_max, color=common.COLOR_DP, linestyle="--", lw=1.8, zorder=4)
32 ax.axvline(epsilon, color="gray", linestyle=":", lw=1.6, zorder=4)
33 ax.axvline(regret_xs_max, color=common.COLOR_RTH, linestyle="-. ", lw=1.6, zorder=4)
34
35 # R_max 标注(箭头从右上文字指向线)
36 ax.annotate(
37     f"$R_{\{max\}}=\{R\_max:.4f\}$\n(鲁棒方案)",
38     xy=(R_max, 0.02), xycoords=("data", "axes fraction"),
39     xytext=(0.34, 0.86), textcoords=("axes fraction", "axes fraction"),
40     fontsize=9.5, color=common.COLOR_DP, ha="center",
41     arrowprops=dict(arrowstyle="->", color=common.COLOR_DP, lw=1.1),
42 )
43 #  $X^*$  后悔值标注
44 ax.annotate(
45     f"$R^{\{max\}}(\mathbf{X}^*)=\{regret\_xs\_max:.4f\}$\n(等权综合最优)",
46     xy=(regret_xs_max, 0.02), xycoords=("data", "axes fraction"),
47     xytext=(0.86, 0.86), textcoords=("axes fraction", "axes fraction"),
48     fontsize=9.5, color=common.COLOR_RTH, ha="center",
```

```

49     arrowprops=dict(arrowstyle="->", color=common.COLOR_RTH, lw=1.1),
50 )
51 #  $\epsilon$  标注(线在左缘,文字放左上,箭头指向左缘)
52 ax.annotate(
53     f"容许上限  $\epsilon$ ",
54     xy=(epsilon, 0.30), xycoords=("data", "axes fraction"),
55     xytext=(0.02, 0.94), textcoords=("axes fraction", "axes fraction"),
56     fontsize=9.5, color="dimgray", ha="left",
57     arrowprops=dict(arrowstyle="->", color="gray", lw=1.1),
58 )
59
60 ax.set_xlabel("候选方案的最大后悔值  $R^{\max}(\mathbf{X})$ ")
61 ax.set_ylabel("候选方案数(共 2000 个)")
62 ax.set_title("候选池最大后悔值分布与鲁棒方案位置")
63 ax.set_xlim(0, max_regret.max() * 1.03)
64
65 fig.tight_layout()
66 common.savefig("fig_regret.png")

```